INTRODUCTION TO RISC-V

RISC-V入门教程

ISA | 汇编指令 | 系统编程 | 组成原理 | 嵌入式应用

数字逻辑基础

主讲 邢建国

中国开放指令生态（RISC-V）联盟 | 浙江中心
China RICS-V Alliance | Zhejiang Center

浙江图灵算力研究院
ZHEJIANG TURING INSTITUTE

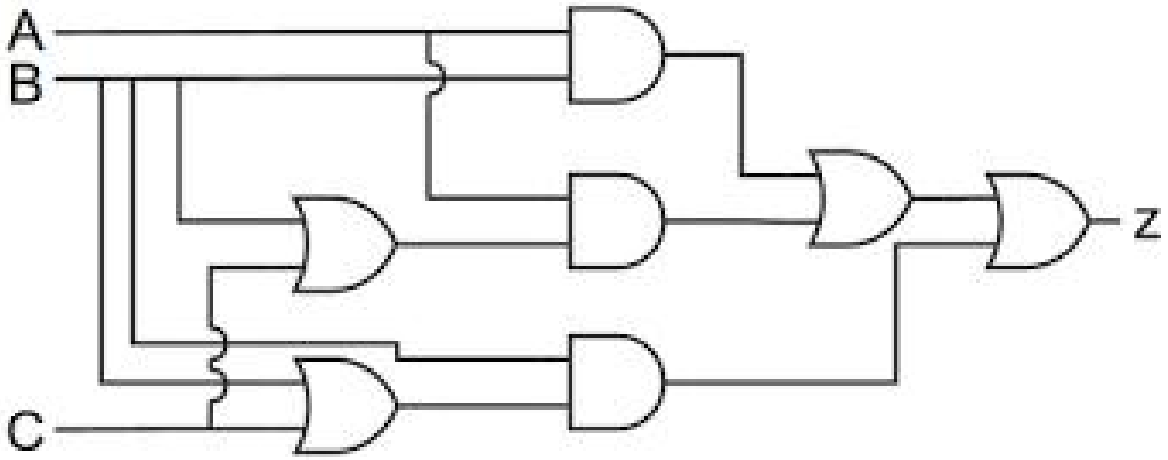
RISC-V®

数字逻辑基础

- 布尔代数
- 数字逻辑
- 组合电路设计
- 时序电路设计

布尔代数

现代数字计算机和数字电路的基础是布尔代数。



■ 乔治●布尔（1815-1864）

An Investigation of the Laws of Thought (1854)

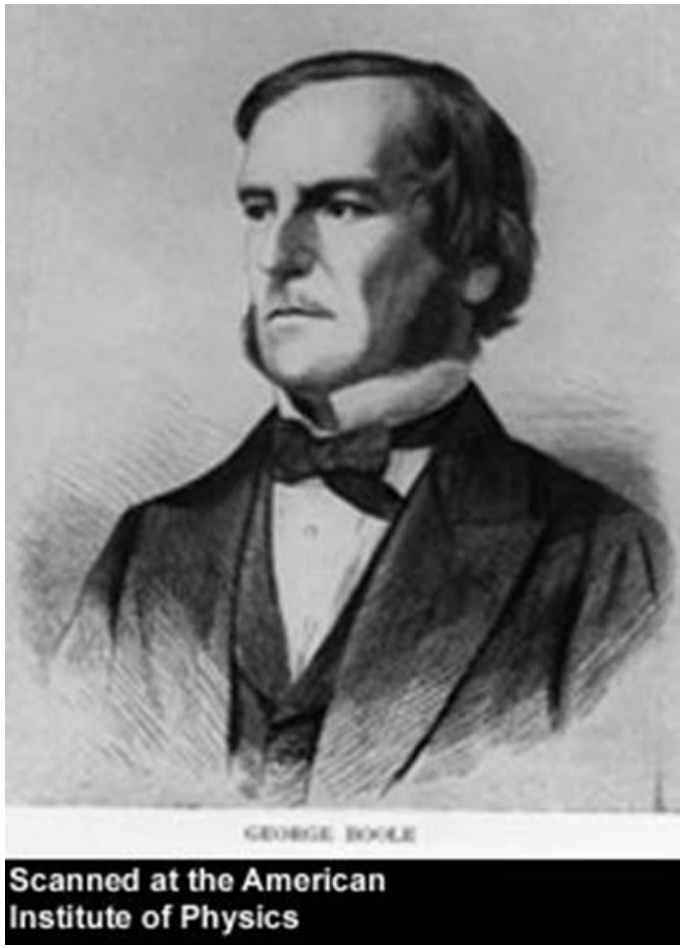
提出二元变量：真/假
三个基本的逻辑运算：AND、OR 和 NOT

建立了逻辑推理和代数运算之间的关系

0 * 0 = 0	假 and 假 = 假
1 * 0 = 0	假 and 真 = 假
0 * 1 = 0	真 and 假 = 假
1 * 1 = 1	真 and 真 = 真

0 + 0 = 0	假 or 假 = 假
1 + 0 = 1	假 or 真 = 真
0 + 1 = 1	真 or 假 = 真
1 + 1 = 1	真 or 真 = 真

$$s = a * \sim b + \sim a * b$$



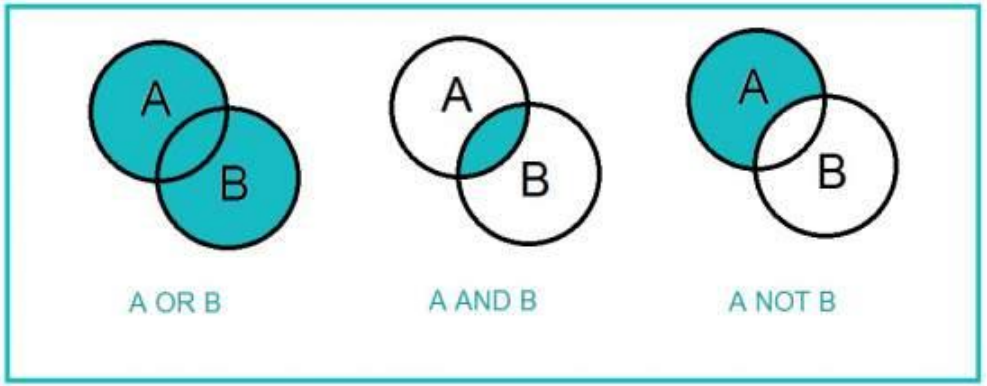
■ 乔治 ● 布尔（1815-1864）

An Investigation of the Laws of Thought (1854)

逻辑和集合的对偶关系

命题逻辑/零阶逻辑

布尔代数规定，个体属于某个集合用1表示，不属于就用0表示。



一名顾客走进宠物店，对店员说：“我想要一只公猫，白色或黄色均可；或者一只母猫，除了白色，其他颜色均可；或者只要是黑猫，我也要。”
店员拿出一只灰色的公猫，请问是否满足要求？
灰色的公猫属于公猫集合，就是1，不属于白色集合，就是0

公猫 X（白色 + 黄色）+ 母猫 X 非白色 + 黑猫

1 X (0 + 0) + 0 X 1 + 0 = 0

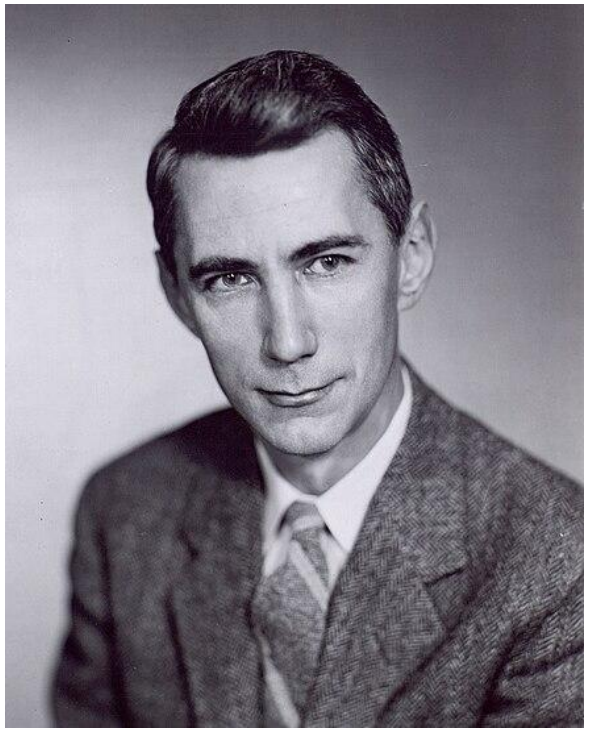
■ 克劳德●香农（1916-2001）

硕士论文：A Symbolic Analysis of Relay and Switching (1937)

证明了（1）布尔代数可用于简化继电器的排列，这些继电器是当时机电自动电话交换机的组成部分。
（2）反过来，使用继电器的排列来解决布尔代数问题也应该是可能的

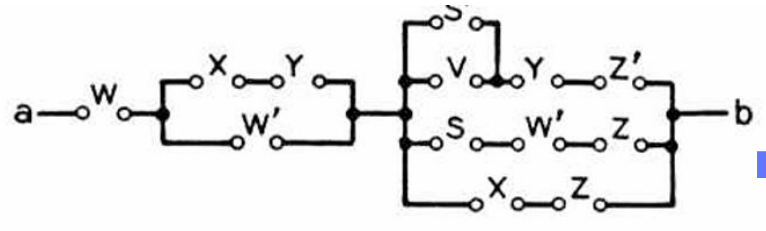
利用电气开关的二进制特性来执行逻辑功能是所有电子数字计算机设计的基本概念。Shannon 的论文在二战期间和之后在电气工程界广为人知，成为实用数字电路设计的基础。

这是有史以来最重要的硕士论文之一，将数字电路设计从一门艺术转变为一门科学。

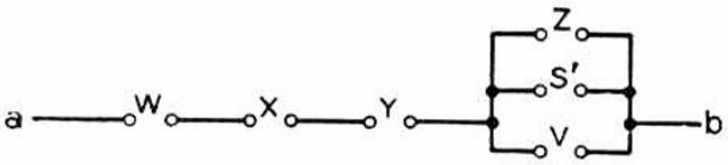


■ 克劳德●香农（1916-2001）

硕士论文：A Symbolic Analysis of Relay and Switching (1937)



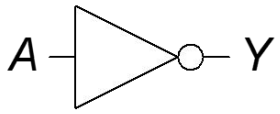
$$
\begin{aligned}
X_{ab} &= W + W'(X + Y) + (X + Z)(S + W' + Z)(Z' + Y + S'V) \\
&= W + X + Y + (X + Z)(S + 1 + Z)(Z' + Y + S'V) \\
&= W + X + Y + Z(Z' + S'V) \ .
\end{aligned}
$$



$$
\begin{aligned}
X_{ab} &= W + X + Y + ZZ' + ZS'V \\
&= W + X + Y + ZS'V \ .
\end{aligned}
$$

基本的逻辑门

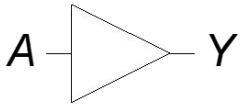
NOT



$Y = \overline{A}$

A	Y
0	1
1	0

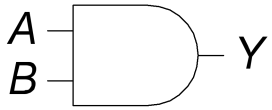
BUF



$Y = A$

A	Y
0	0
1	1

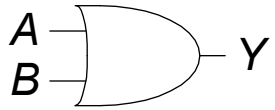
AND



$Y = AB$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR

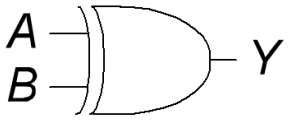


$Y = A + B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

基本的逻辑门

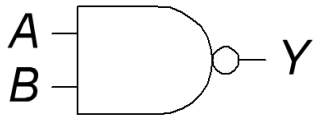
XOR



$Y = A \oplus B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

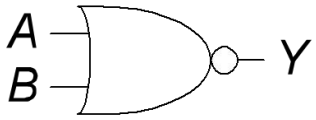
NAND



$Y = \overline{AB}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

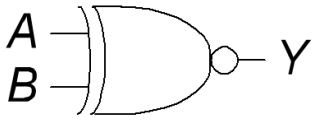
NOR



$Y = \overline{A + B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR

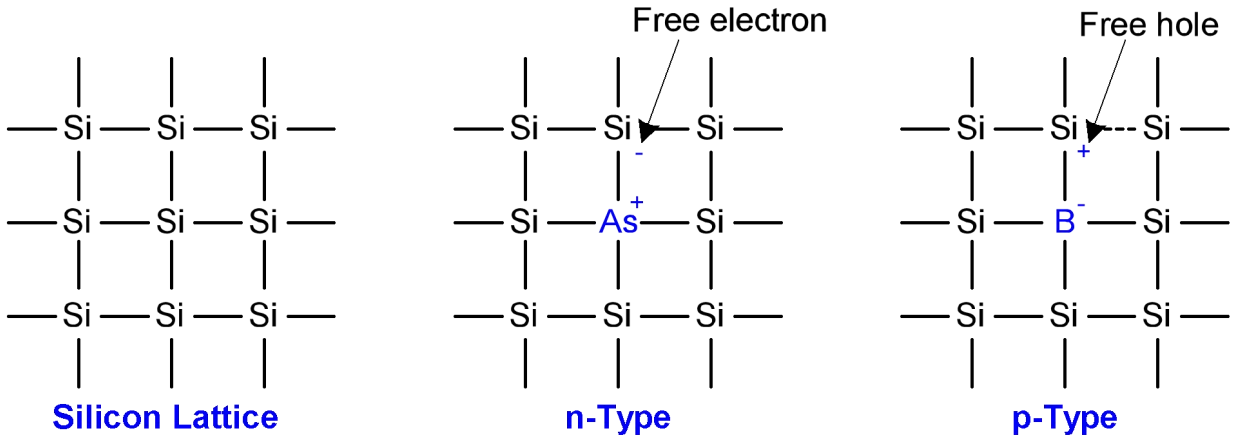


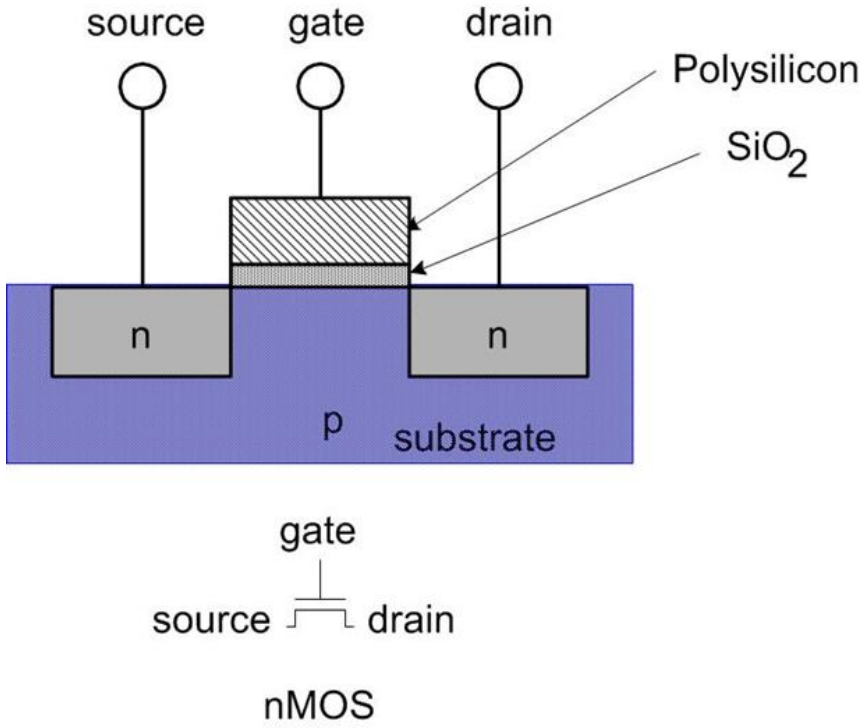
$Y = \overline{A \oplus B}$

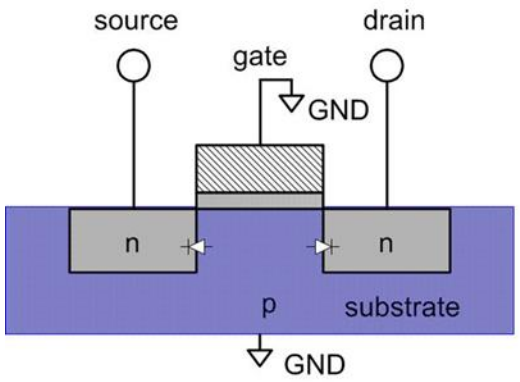
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

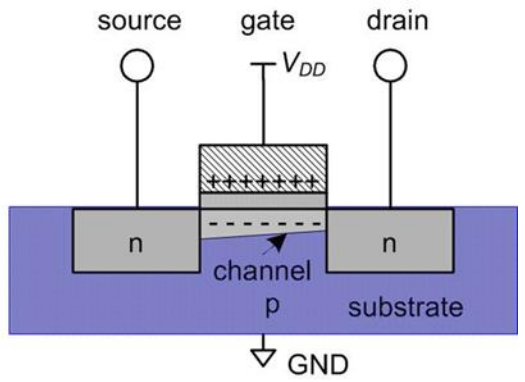
MOS开关

金属氧化物硅（MOS）晶体管：
M：多晶硅（以前是金属）栅
O：氧化物（二氧化硅）绝缘体
S：掺杂硅

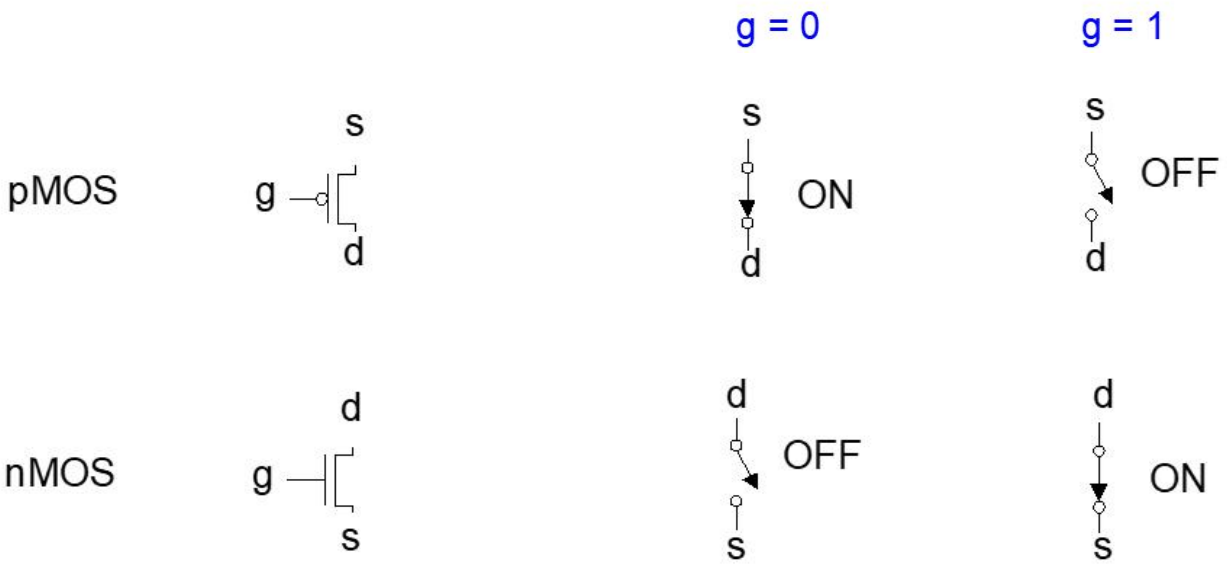


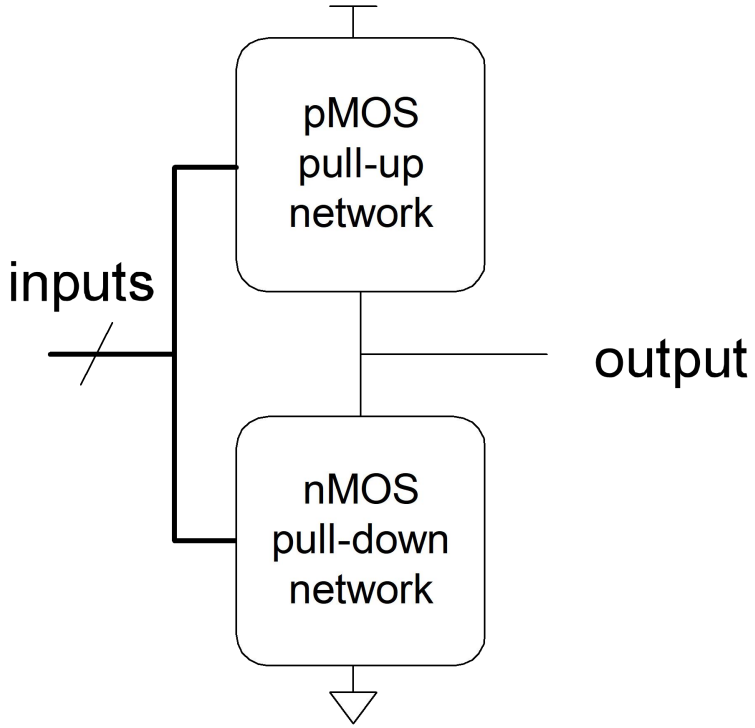






NMOS、PMOS、CMOS





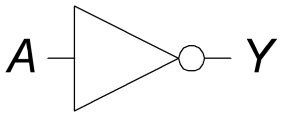
pMOS：通常用于传递高电平信号（空穴），因此将源极(s)连接VDD

nMOS：通常用于传递低电平信号（电子），因此将源极(s)连接到 GND

CMOS：pMOS + nMOS

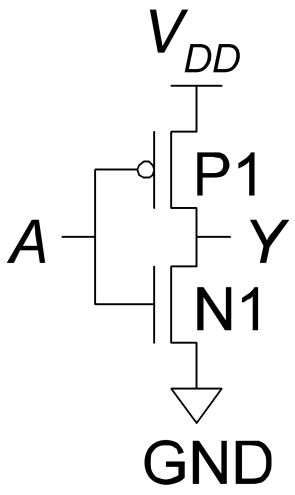
CMOS基本逻辑门：非门

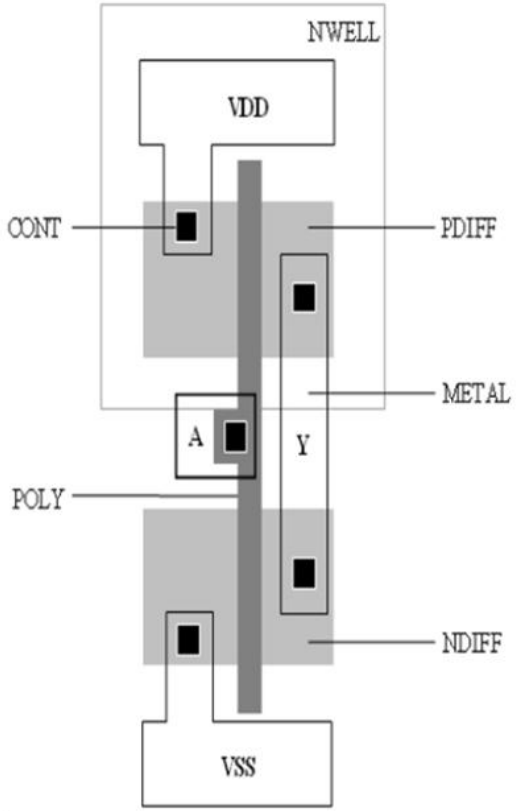
NOT

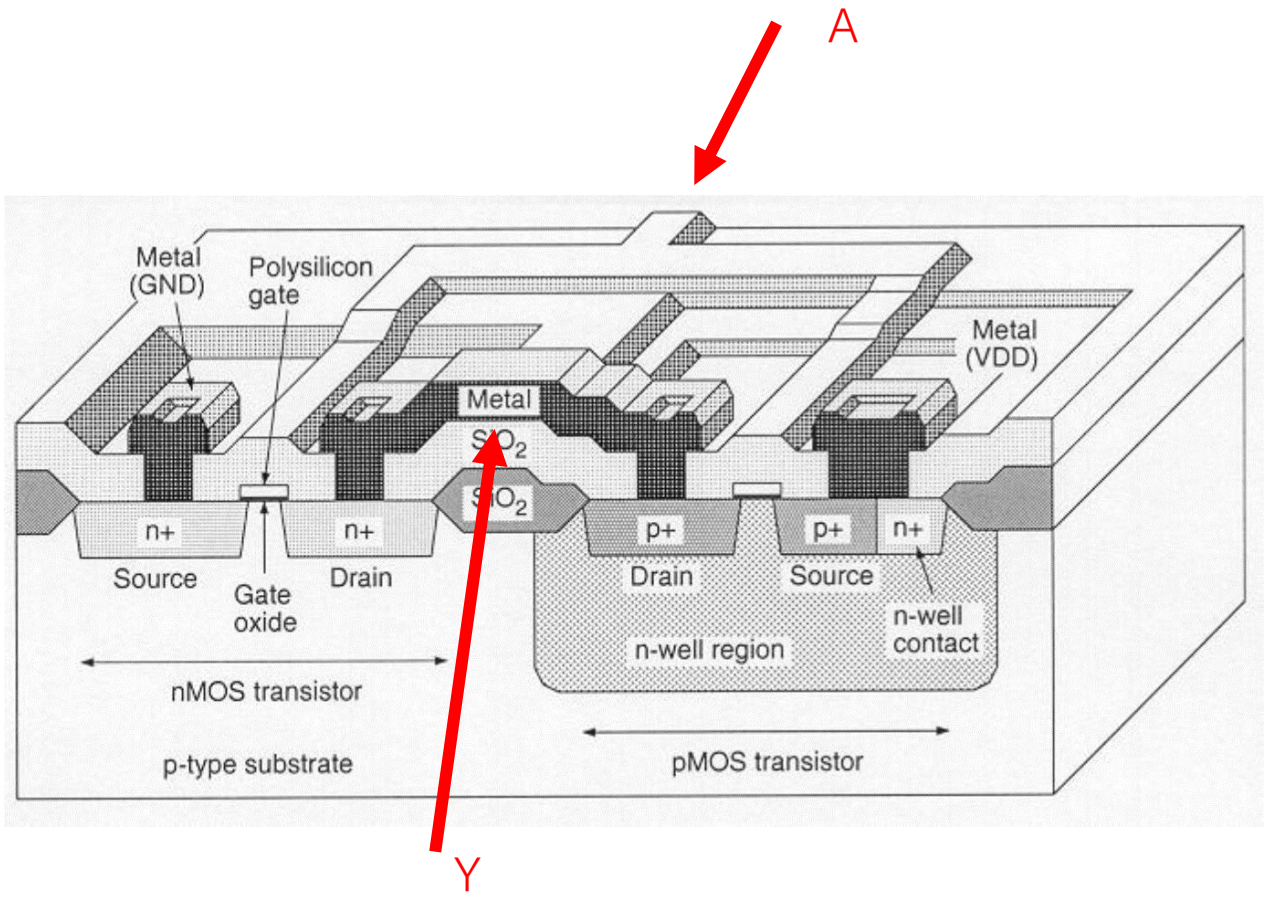


$Y = \overline{A}$

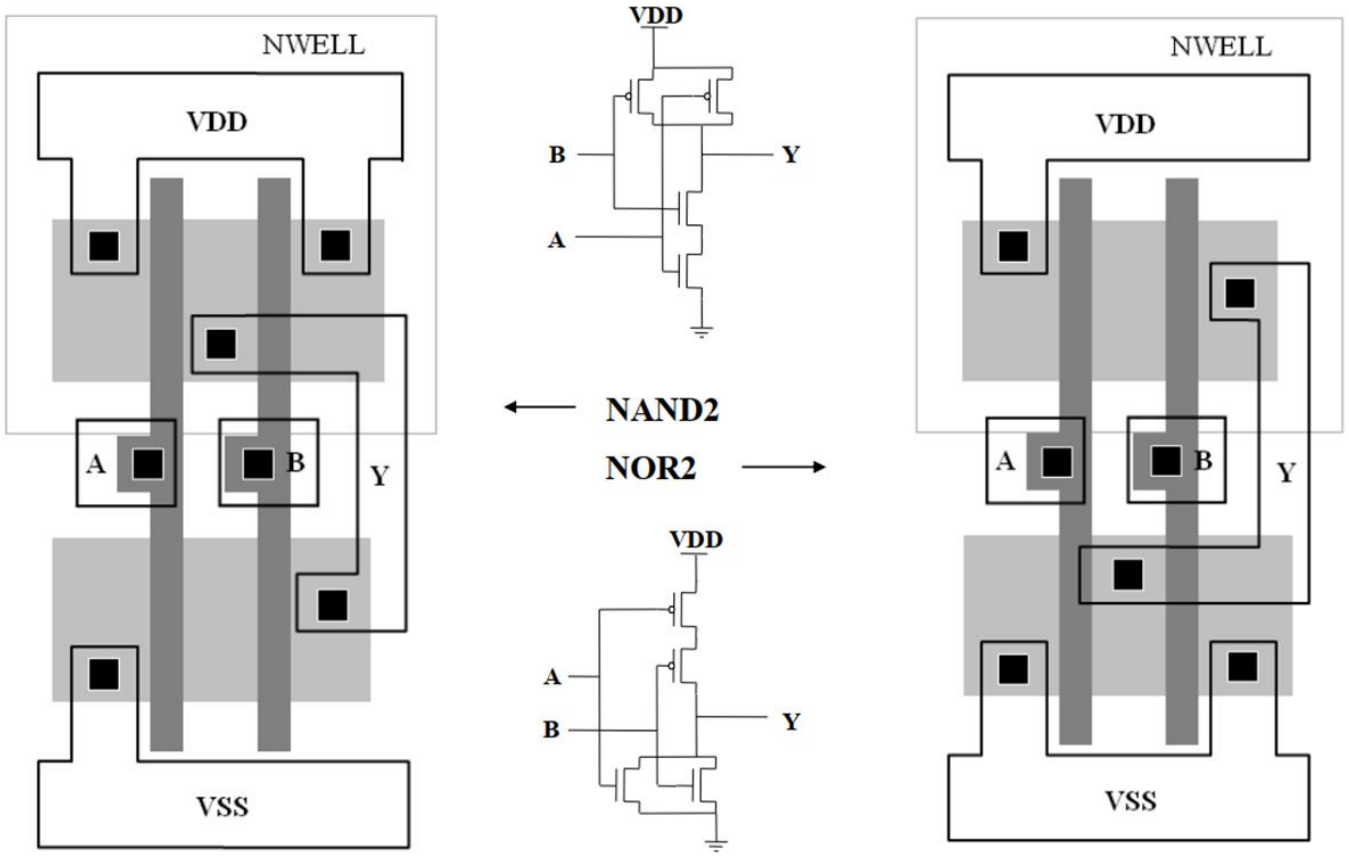
A	Y
0	1
1	0

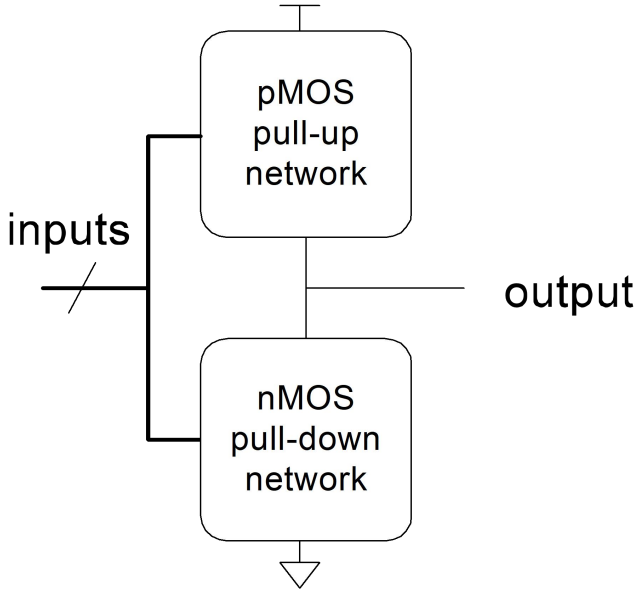






CMOS基本逻辑门：与非门、或非门



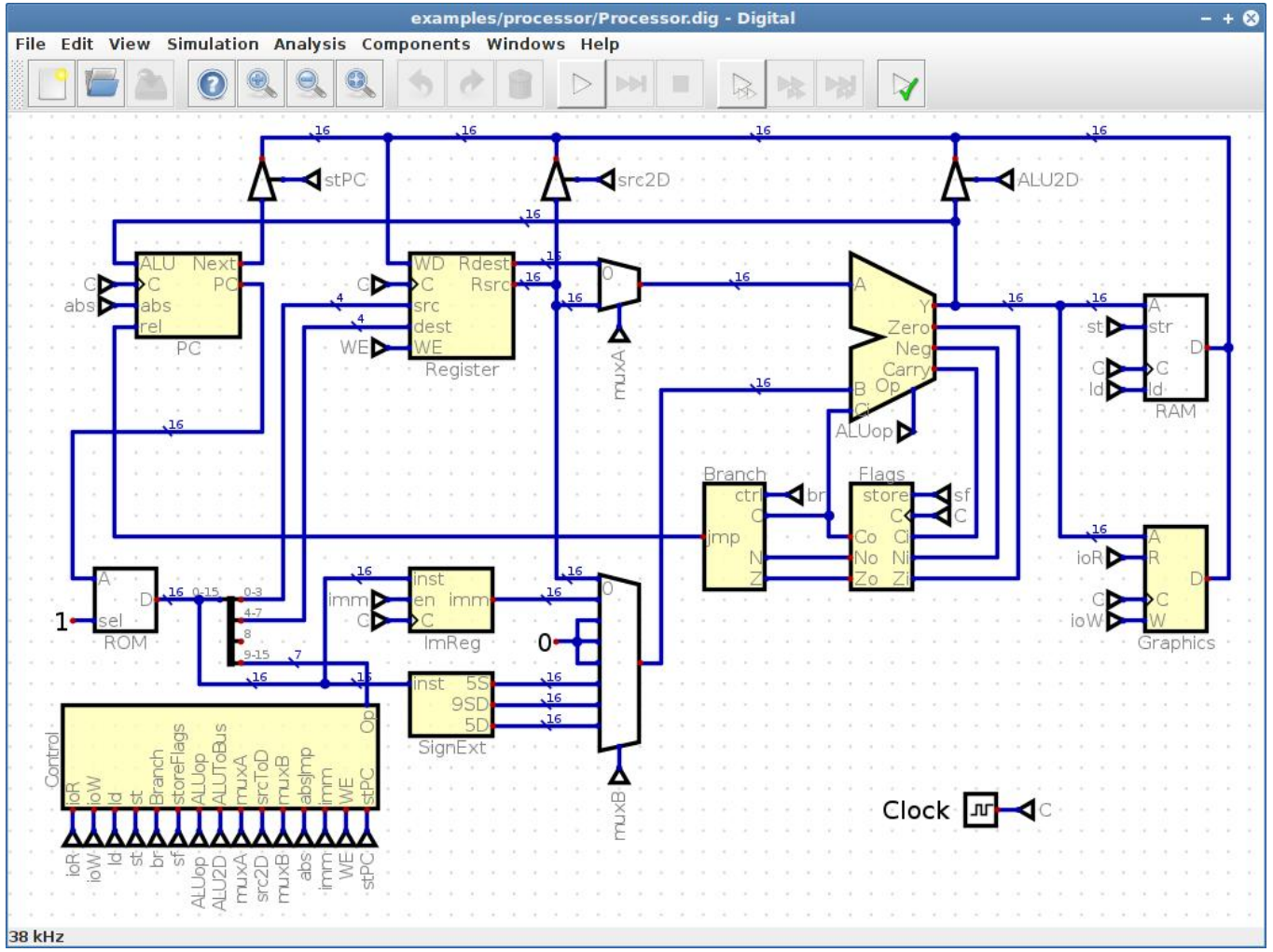


Digital：数字逻辑仿真软件

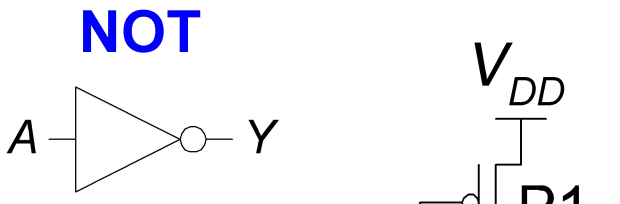
Digital是一款开源的、易于使用的数字逻辑设计器和电路仿真器，专为教育目的而设计

https://github.com/hneemann/Digital

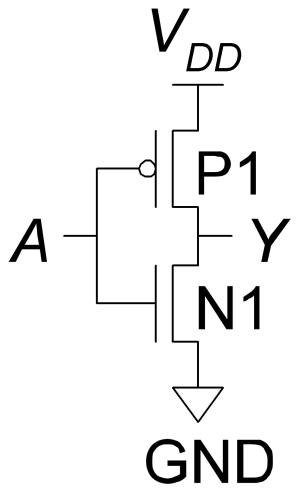
- 安装：
  - 下载最新版digital.zip
- 运行:
  - 解压后运行digital.exe（Windows）
  - 如果提示缺少java运行环境（"This application requires a Java Runtime Environment 1.8.0），则按提示下载jre8或更高版本

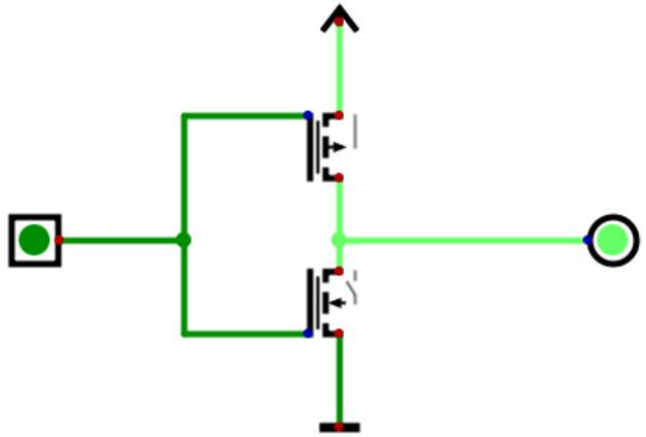


CMOS组成基本逻辑门

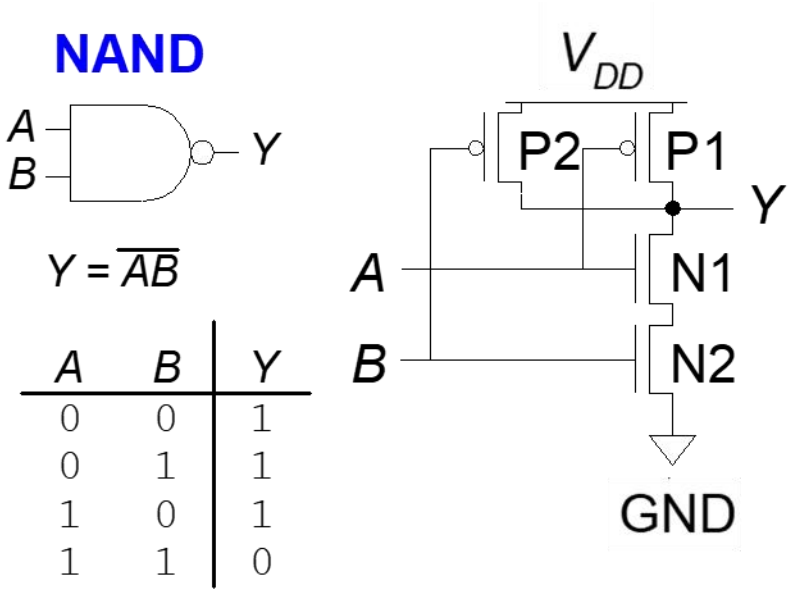


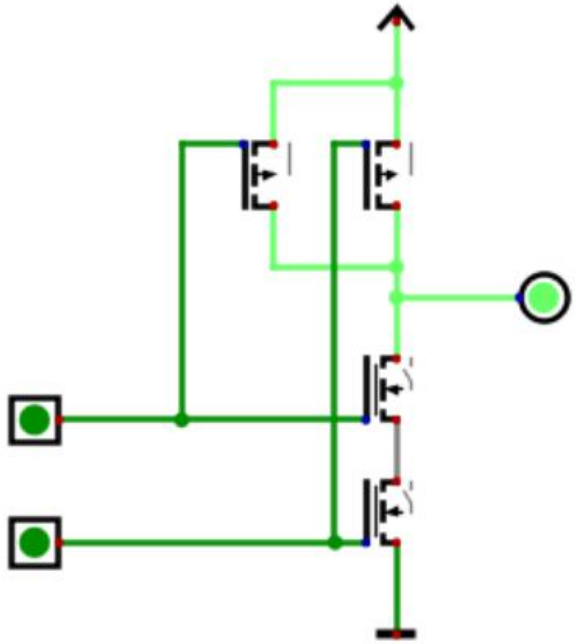
A	Y
0	1
1	0



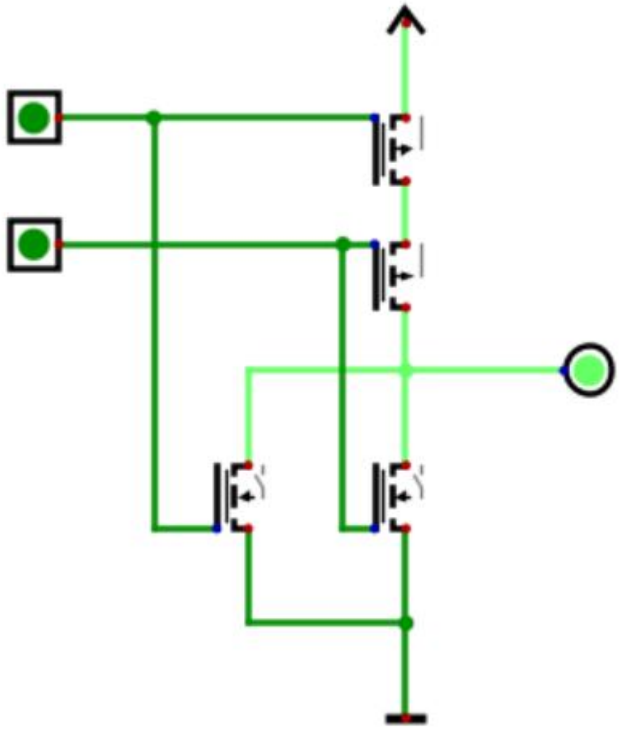


CMOS组成基本逻辑门——与非门

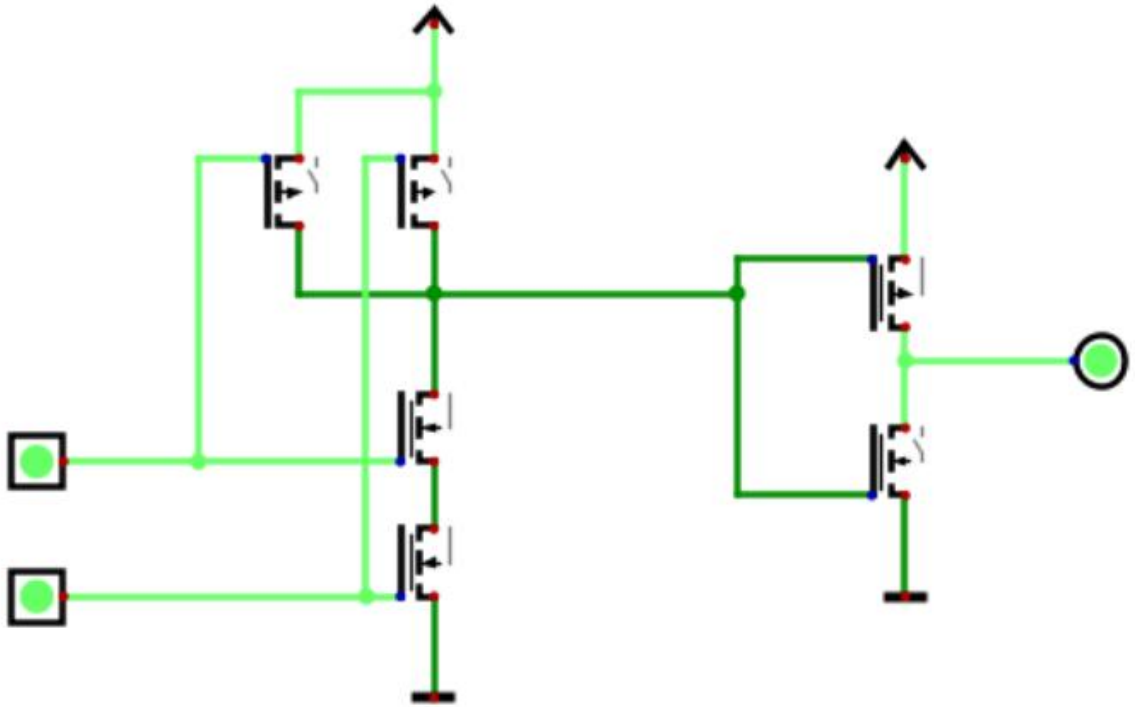




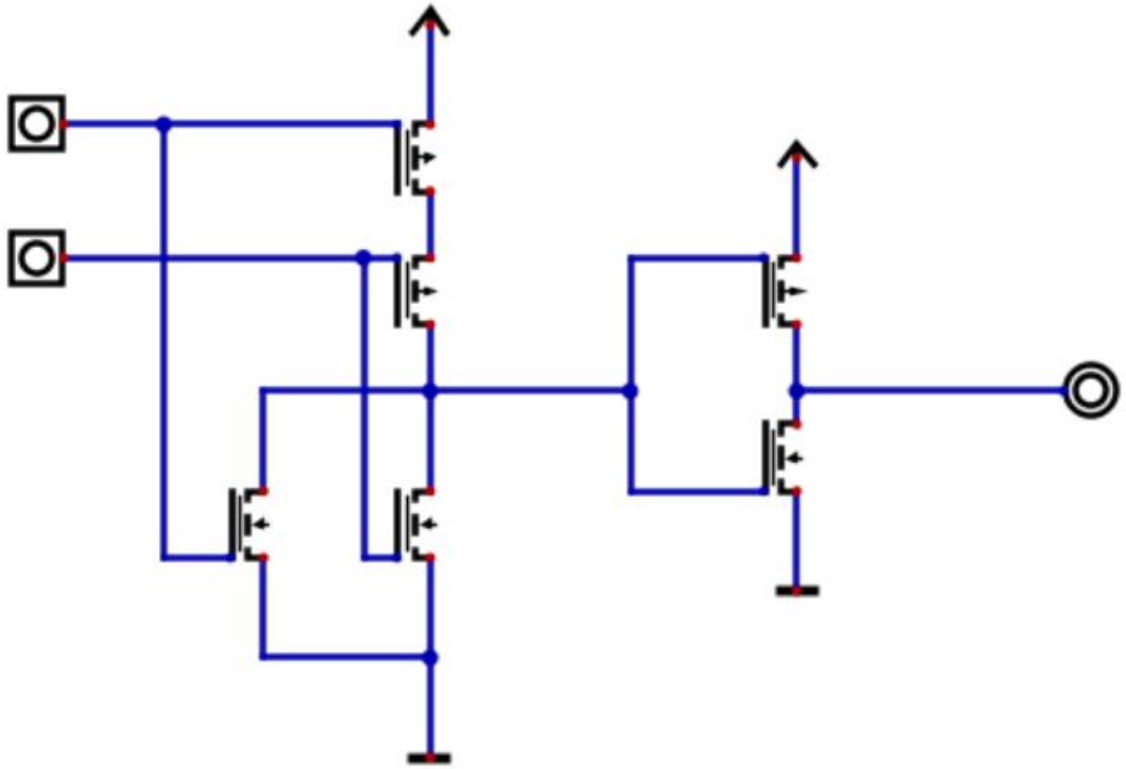
■ CMOS组成基本逻辑门 — —或非门



■ CMOS组成基本逻辑门 — —与门



■ CMOS组成基本逻辑门 — 一或门



布尔代数：公理

Number	Axiom	Name
A1	$B = 0$ if $B \neq 1$	Binary Field
A2	$\overline{0} = 1$	NOT
A3	$0 \bullet 0 = 0$	AND/OR
A4	$1 \bullet 1 = 1$	AND/OR
A5	$0 \bullet 1 = 1 \bullet 0 = 0$	AND/OR

■ 布尔代数公理（对偶性）

Number	Axiom	Dual	Name
A1	B = 0 if B ≠ 1	B = 1 if B ≠ 0	Binary Field
A2	$\overline{0} = 1$	$\overline{1} = 0$	NOT
A3	0 • 0 = 0	1 + 1 = 1	AND/OR
A4	1 • 1 = 1	0 + 0 = 0	AND/OR
A5	0 • 1 = 1 • 0 = 0	1 + 0 = 0 + 1 = 1	AND/OR

- 、 + 互换

0 、1 互换

布尔代数定理

Number	Theorem	Dual	Name
T1	$B \bullet 1 = B$	$B + 0 = B$	Identity
T2	$B \bullet 0 = 0$	$B + 1 = 1$	Null Element
T3	$B \bullet B = B$	$B + B = B$	Idempotency
T4	$\overline{\overline{B}} = B$		Involution
T5	$B \bullet \overline{B} = 0$	$B + \overline{B} = 1$	Complements

- $\bullet$ 、 $+$ 互换

0 、1 互换

布尔代数定理

#	Theorem	Dual	Name
T6	$B \bullet C = C \bullet B$	$B+C = C+B$	Commutativity
T7	$(B \bullet C) \bullet D = B \bullet (C \bullet D)$	$(B + C) + D = B + (C + D)$	Associativity
T8	$B \bullet (C + D) = (B \bullet C) + (B \bullet D)$	$B + (C \bullet D) = (B+C)\ (B+D)$	Distributivity
T9	$B \bullet (B+C) = B$	$B + (B \bullet C) = B$	Covering
T10	$(B \bullet C) + (B \bullet \overline{C}) = B$	$(B+C) \bullet (B+\overline{C}) = B$	Combining
T11	$(B \bullet C) + (\overline{B} \bullet D) + (C \bullet D) =$ $(B \bullet C) + (\overline{B} \bullet D)$	$(B+C) \bullet (\overline{B}+D) \bullet (C+D) =$ $(B+C) \bullet (\overline{B}+D)$	Consensus

德摩根定理

#	Theorem	Dual	Name
T12	$\overline{B \bullet C \bullet D \ldots} = \overline{B} + \overline{C} + \overline{D} \ldots$	$\overline{B + C + D \ldots} = \overline{B} \bullet \overline{C} \bullet \overline{D} \ldots$	De Morgan's Theorem

布尔表达式化简

$$Y = A\bar{B}C + ABC + \bar{A}BC$$

$$= A\bar{B}C + ABC + ABC + \bar{A}BC \quad \text{T3': Idempotency}$$

$$= (A\bar{B}C + ABC) + (ABC + \bar{A}BC) \quad \text{T7': Associativity}$$

$$= AC + BC \quad \text{T10: Combining}$$

$$Y = A(AB + ABC) \Rightarrow$$

$$Y = A'BC + A' \Rightarrow$$

SOP（Sum of Products）和POS（Products of Sum）

SOP – sum-of-products

C	M	E	minterm
0	0	0	$\overline{C}\ \overline{M}$
0	1	0	$\overline{C}\ M$
1	0	1	$C\ \overline{M}$
1	1	0	$C\ M$

POS – product-of-sums

C	M	E	maxterm
0	0	0	$C + M$
0	1	0	$C + \overline{M}$
1	0	1	$\overline{C} + M$
1	1	0	$\overline{C} + \overline{M}$

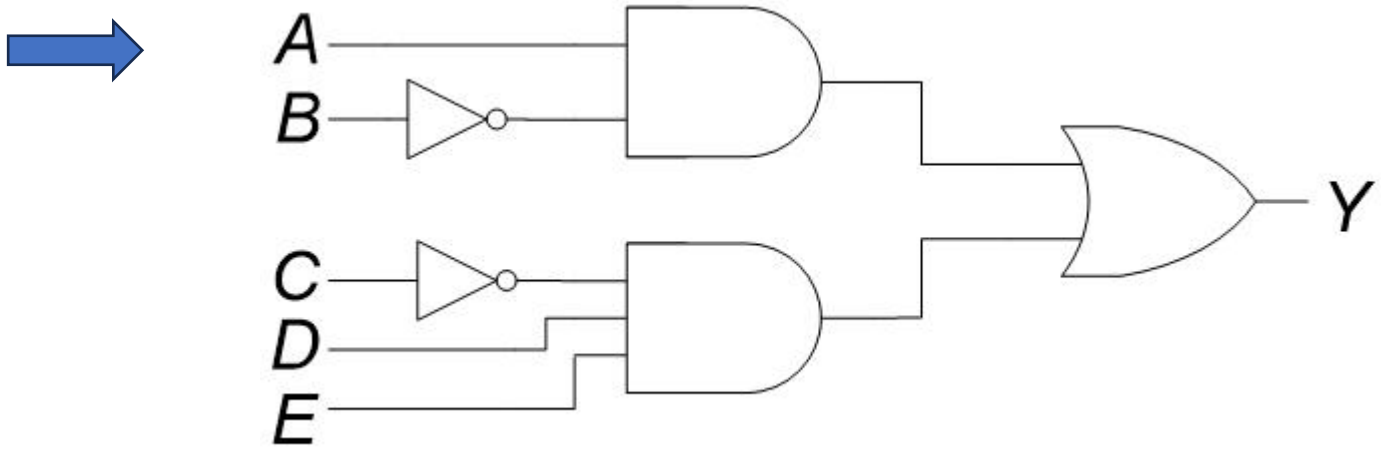
$$E = C\overline{M}$$

$$E = (C + M)(C + \overline{M})(\overline{C} + \overline{M})$$

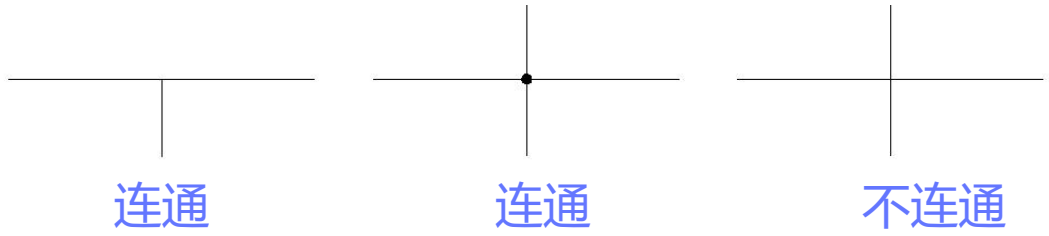
等价

用逻辑门实现布尔表达式

$$Y = A\bar{B} + \bar{C}DE$$

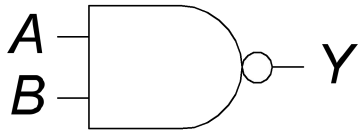


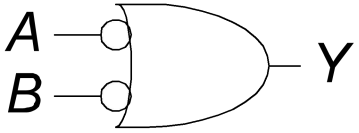
- 输入放在左边
- 输出放在右边
- 逻辑门从左向右放置
- 连线尽量用直线



用逻辑门实现布尔表达式：德摩根定理

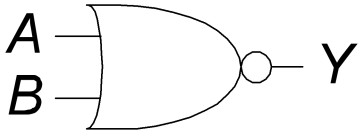
$$Y = \overline{AB} = \overline{A} + \overline{B}$$

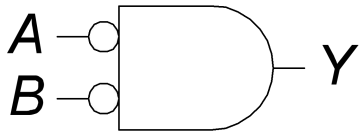




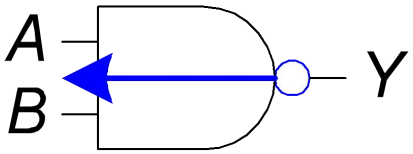
与非门两种形式

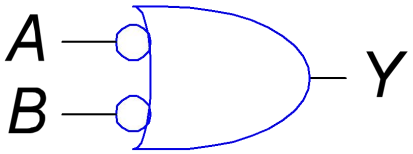
$$Y = \overline{A + B} = \overline{A} \bullet \overline{B}$$





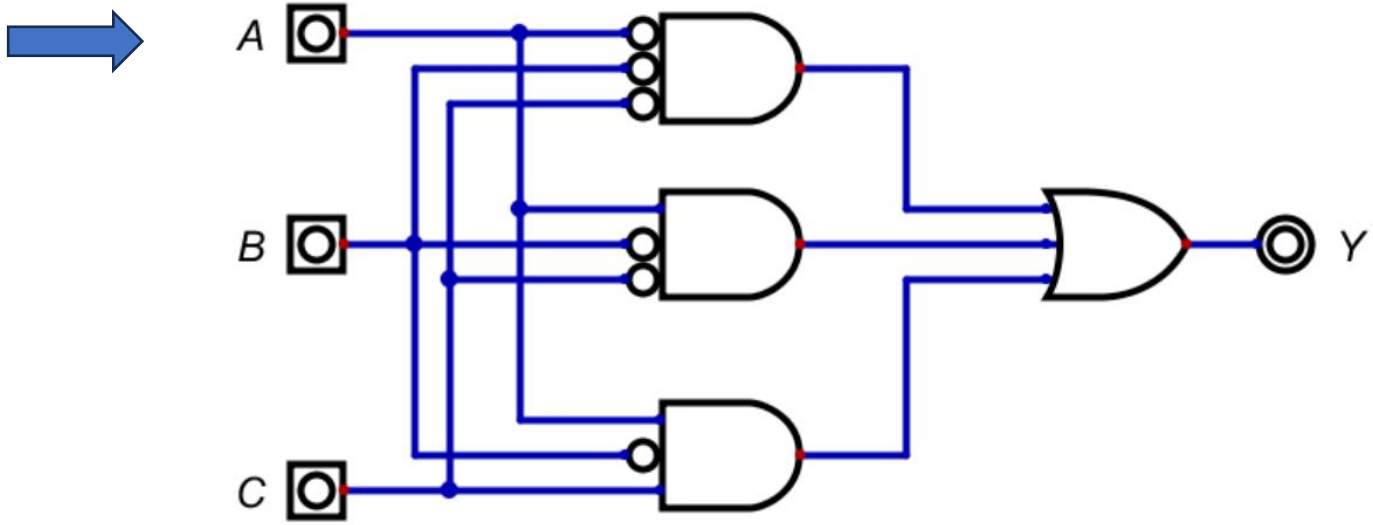
或非门两种形式



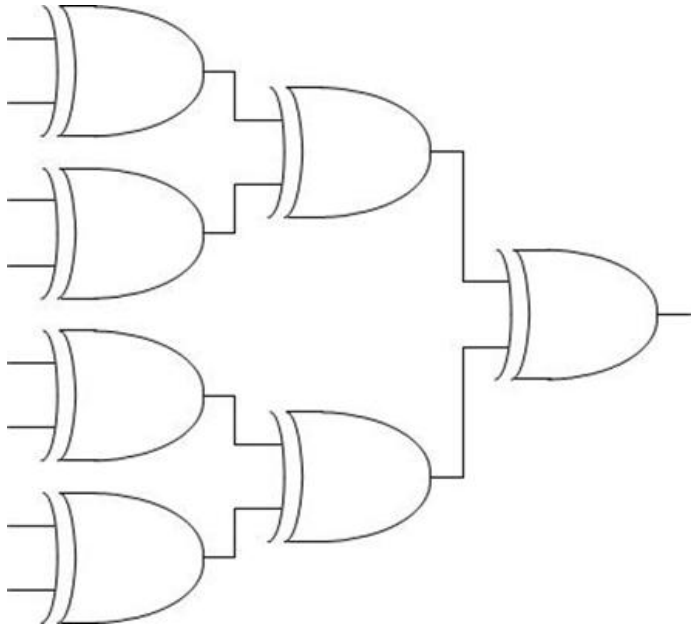


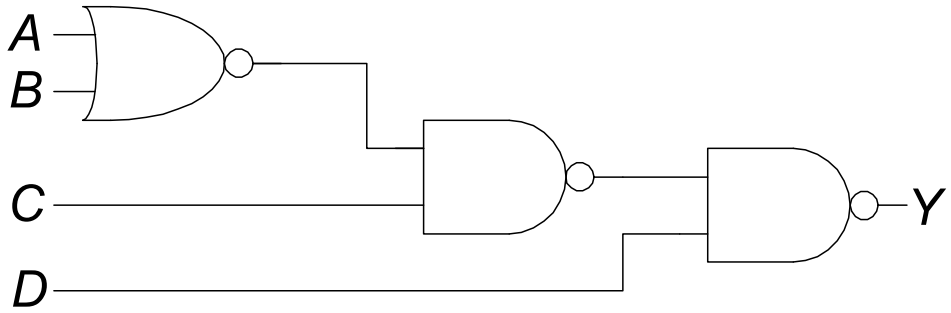
■ 用逻辑门实现布尔表达式（两层逻辑：与门+或门）

$$Y = \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + A\bar{B}C$$



更复杂的逻辑



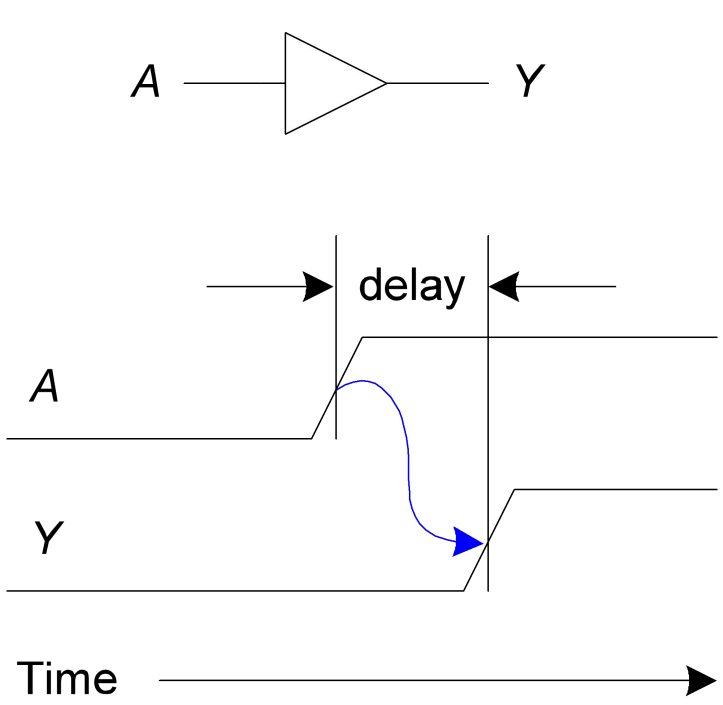


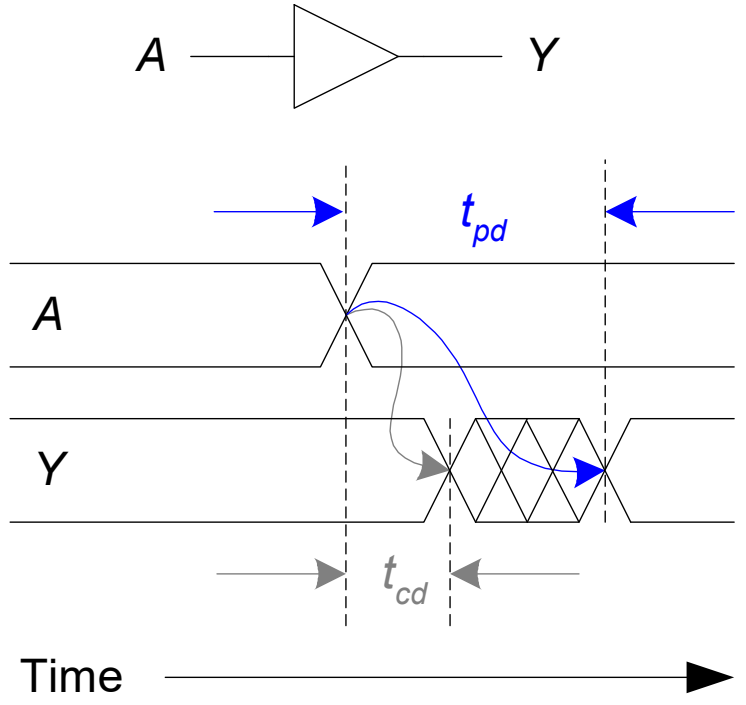
时序和延时

延时：从输入变化和输出变化之间的时间（取决于线路的阻抗、信号传输速度、传输路径）

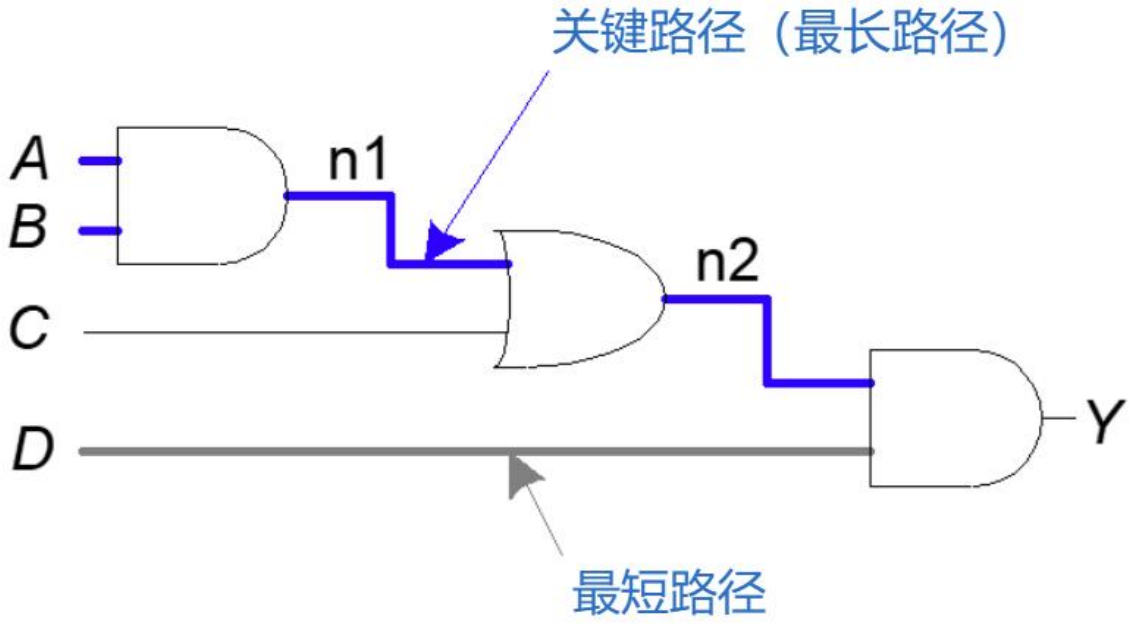
传播延时：t_{pd}=max(从输入到输出的延时)

污染延时：t_{cd} =min(从输入到输出的延时)





关键路径



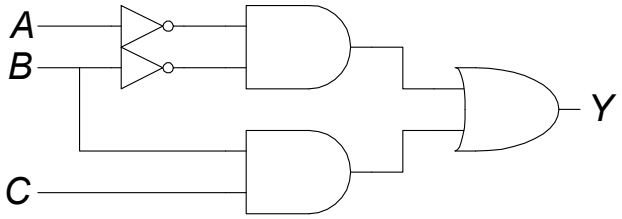
关键路径（最长路径）：$t_{pd} = 2t_{pd\_AND} + t_{pd\_OR}$ **(max delay)**

最短路径：$t_{cd} = t_{cd\_AND}$ **(min delay)**

毛刺(glitches)

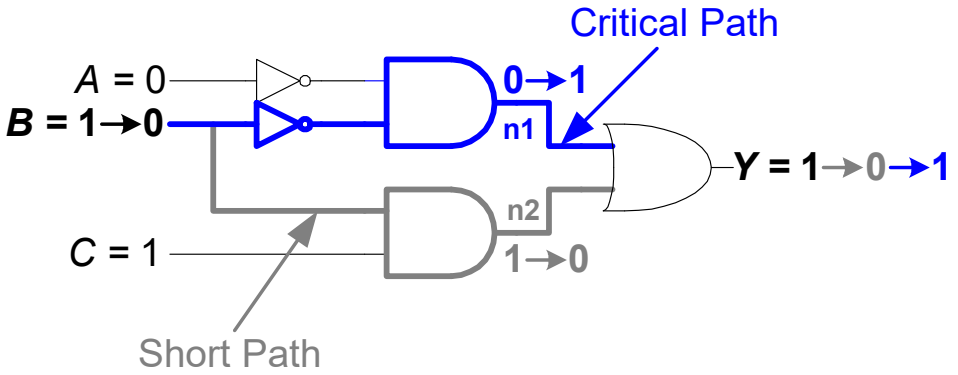
毛刺：一个输入信号的改变引起输出信号的多次变化（信号传输路径不同导致）

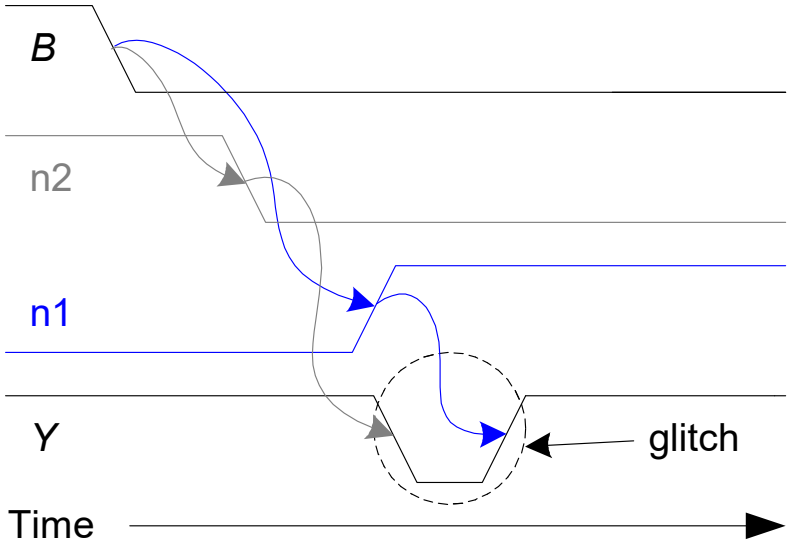
当A=0，C=1，B由高到低时，输出如何变化?



Y	AB=00	AB=01	AB=11	AB=10
C=0	1	0	0	0
C=1	1	1	1	0

$Y = \bar{A}\bar{B} + BC$

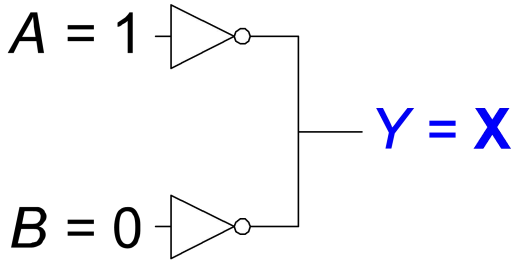




X（未知态）和Z（高阻态）

X代表未知状态（Unknown）

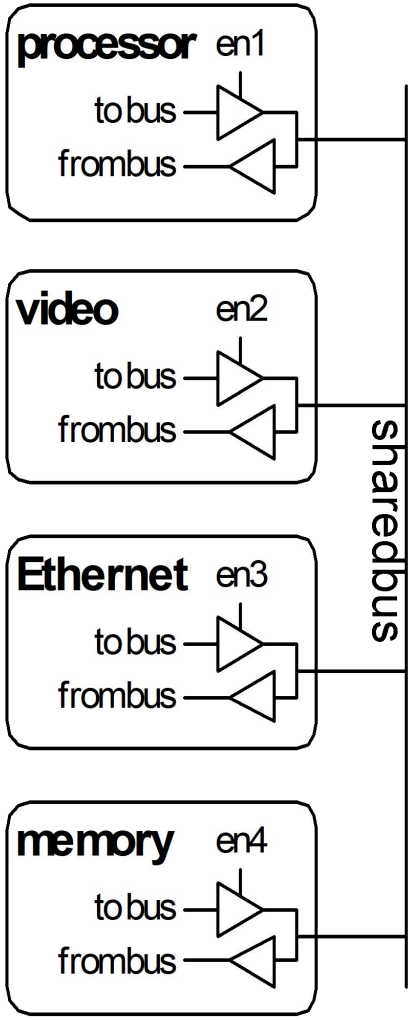
- 发生了不能解决的逻辑冲突
  - 实际值可能为0，1，或在0和1之间的禁区
- 仿真时没有初始化
- Don’t care



X（未知态）和Z（高阻态）

Z代表高阻态（ High Impedance ）

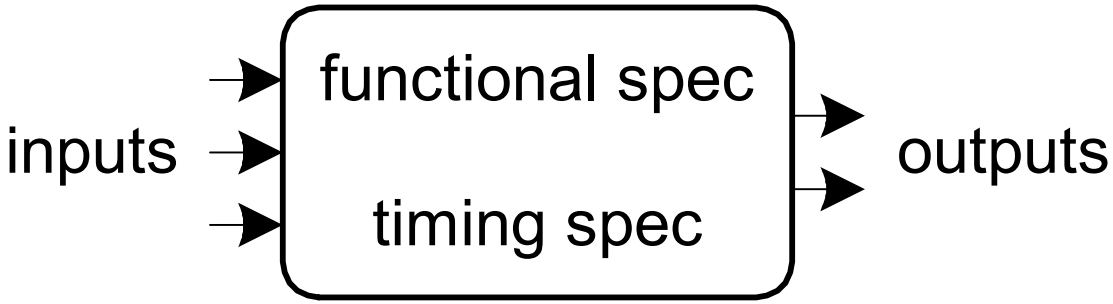
- 又称为HiZ、Tri-State或Disabled Driver状态。
- 表示该信号没有被其他信号驱动，处于一个悬空状态(float)
- 实际电压值可能为0，1或其他值
  - 电压表无法判断一个节点是否处于高阻态
- 可用于连接多个输入驱动的共享总线



数字逻辑电路

数字逻辑电路（数字系统）

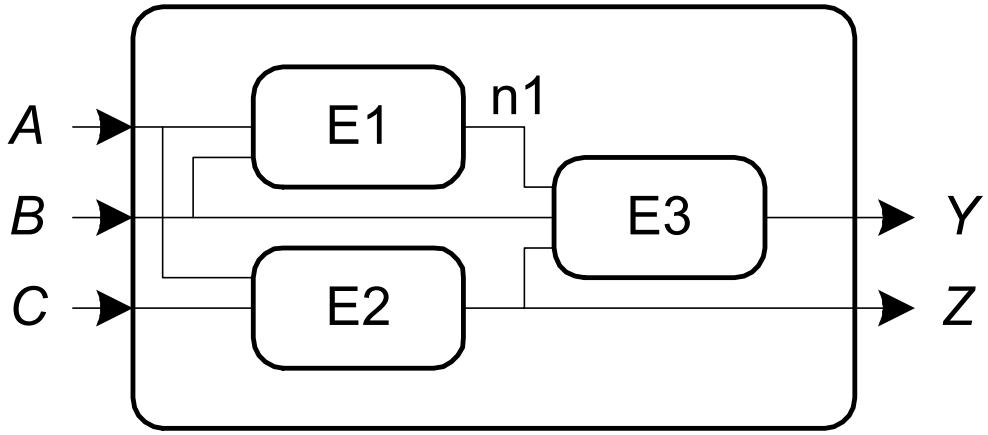
- 输入
- 输出
- 功能规格
- 时序规格



数字逻辑电路

数字逻辑电路

- 节点
  - 输入：A，B，C
  - 输出：Y，Z
  - 内部节点：n1
- 部件
  - E1、E2、E3
  - 整个系统

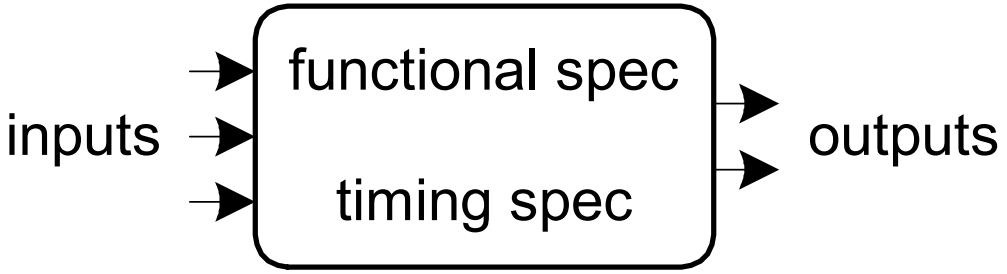


$$Y = f(A, B, C)$$
$$Z = g(A, B, C)$$

数字逻辑电路

数字逻辑电路

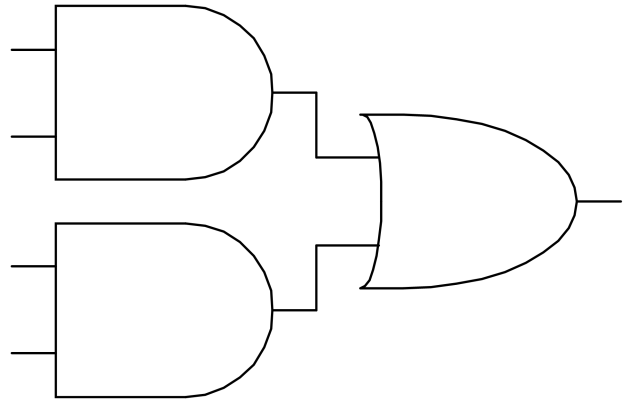
- 组合逻辑电路
  - 无记忆
  - 输出只与输入值有关
- 时序逻辑电路
  - 有记忆
  - 输出不仅与当前输入有关，还和过去的输入有关



组合逻辑

- 无记忆
- 输出只与输入值有关

- 组成规律
  - 每个部件都是组合逻辑
  - 每个节点要么是一个输入节点，要么只和一个输出连接
  - 电路中没有回路

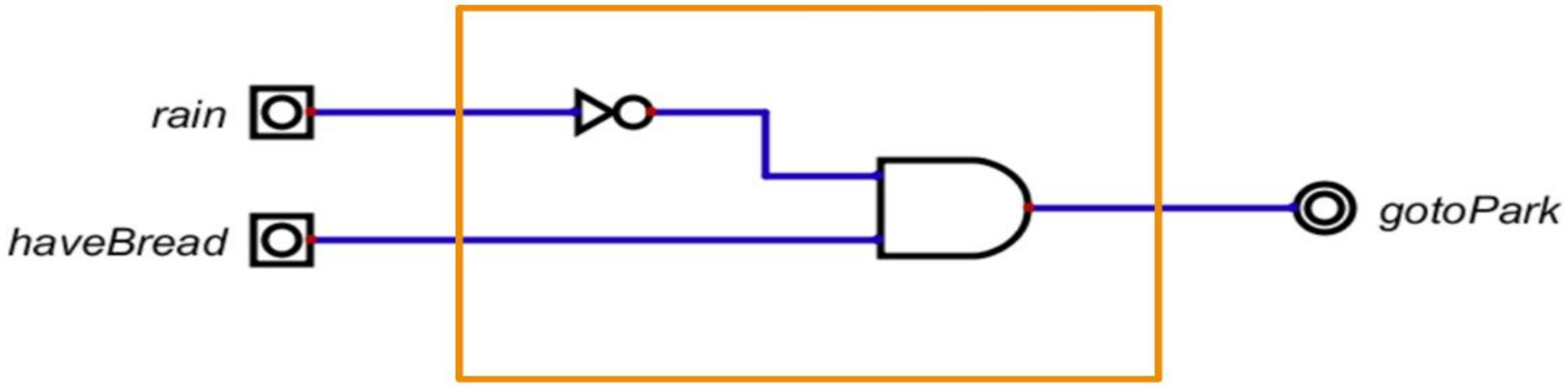


组合逻辑示例

- 如果不下雨，并且有面包，则去公园
- 输入：rain，haveBread
- 输出：gotoPark
- 功能描述：

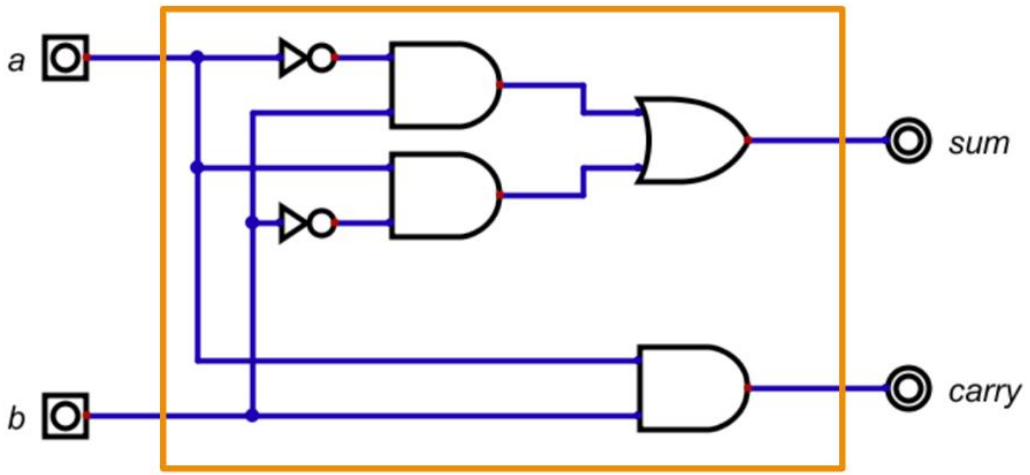
  gotoPark = f (rain, haveBreak)

- (~ rain) & haveBread



1位半加器（HalfAdder）

- 输入：a, b　　（a和b为1位二进制数）
- 输出：sum, carry　　（sum和carry为1位二进制数）
- 功能描述：
- { carry, sum } = a + b



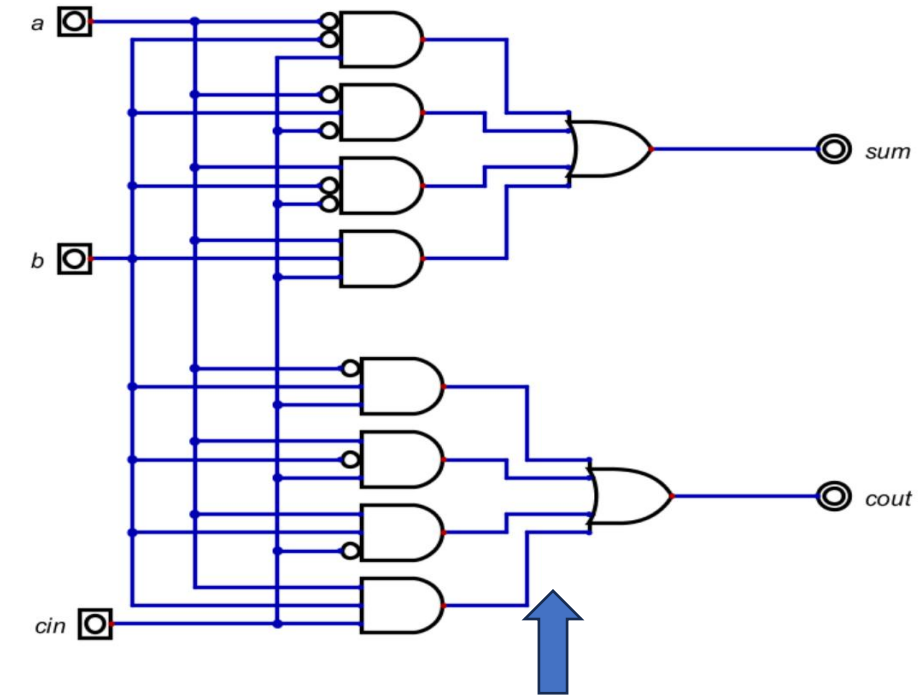
a	b	sum	carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

sum = (~a & b) | (a & ~b)
carry = a & b

1位全加器（FullAdder）

- 输入：a, b, cin
- 输出：sum, cout
- 功能描述：　　{ cout, sum } = a + b + cin

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1




```
sum  = (~a & ~b & ~cin) |
       (~a &  b & ~cin) |
       ( a & ~b & ~cin) |
       ( a &  b &  cin)

cout = (~a &  b &  cin) |
       ( a &  b &  cin) |
       ( a &  b &  cin) |
       ( a &  b &  cin)
```

1位全加器 (FullAdder)

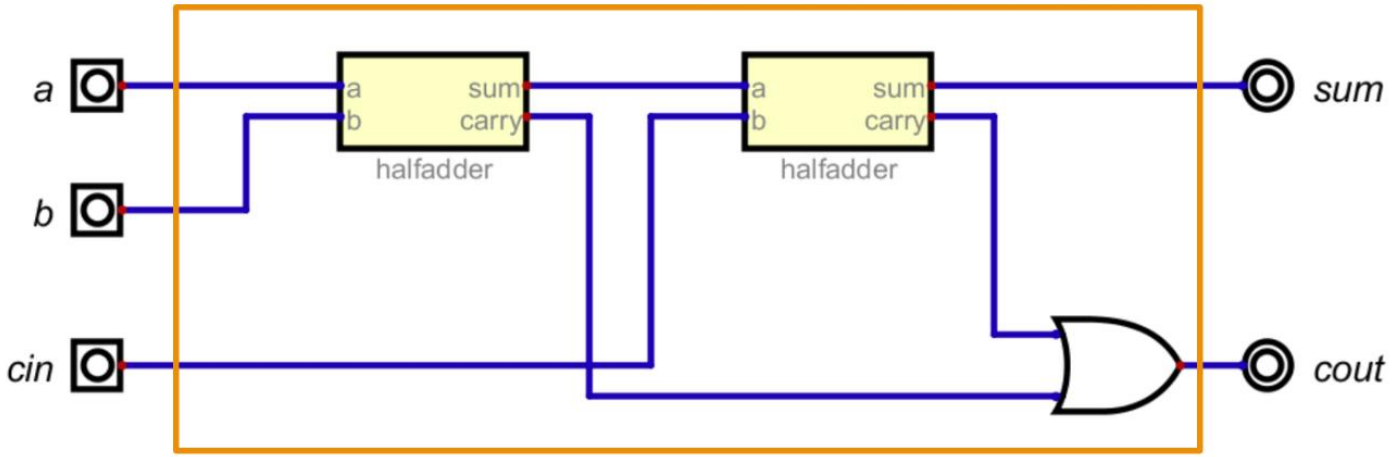
- 输入：a, b, cin
- 输出：sum, cout
- 功能描述：　　{ cout, sum } = a + b + cin

- 使用已有的半加器：

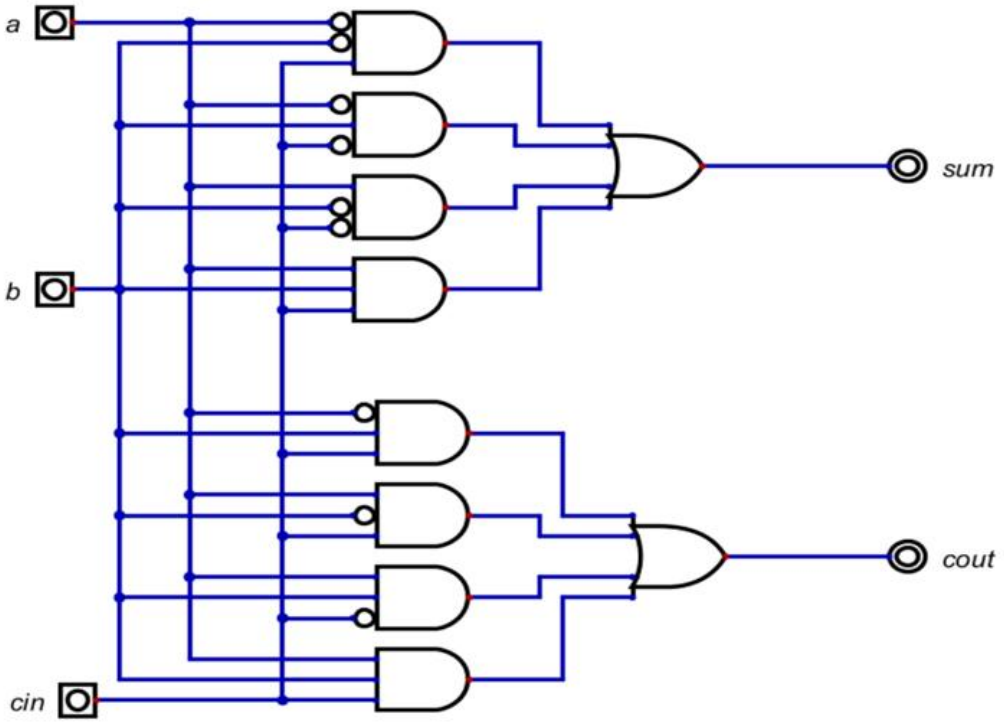
  {s1, c1}　= halfadder(a, b)

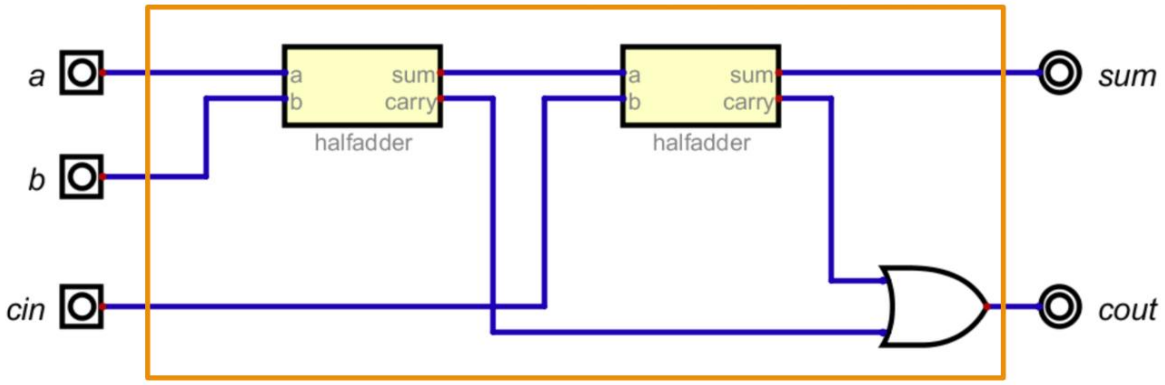
  {sum,c2} = halfadder(s1, cin)

  cout = c1 | c2



1位全加器（FullAdder）





电路统计

组件	输入	位	地址位	数量
And	3	1		8
Or	4	1		2

最长路径包含 2 个门电路

电路统计

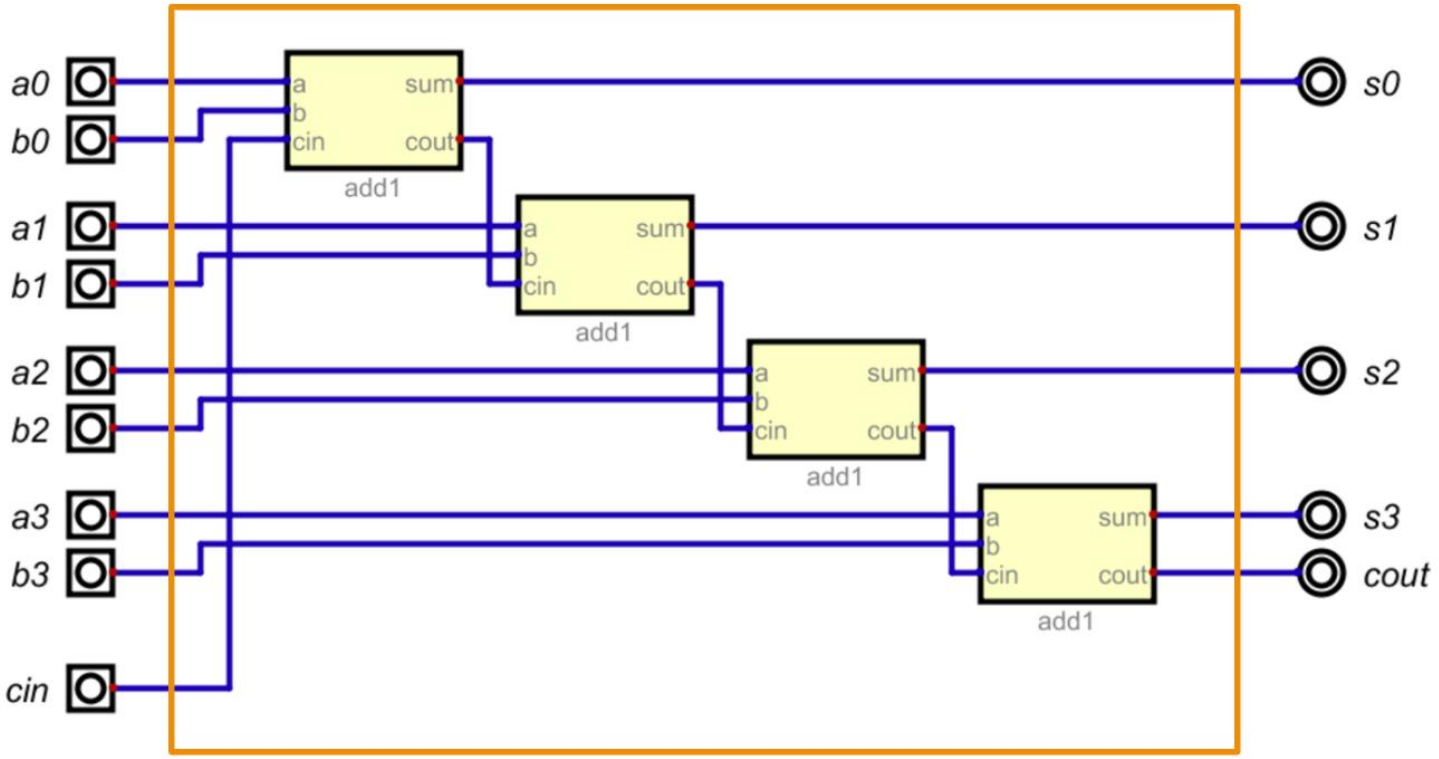
组件	输入	位	地址位	数量
And	2	1		6
Not	1	1		4
Or	2	1		3

最长路径包含 5 个门电路

■ 4位加法

- 输入：a0, a1, a2, a3, b0, b1, b2, b3, cin
- 输出：s0, s1, s2, s3, cout
- 功能描述：{ cout, sum } = a + b + cin

- 使用4个全加器级联
  - 串行进位加法器
  - (ripple carry adder)



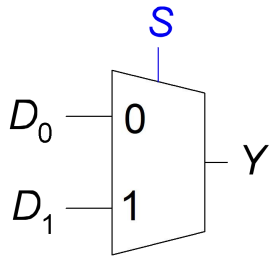
8位加法、16位加法、32位加法…

- 8位加法器
  - 8个1位加法器级联
  - 2个4位加法器级联
- 16位加法器
  - 2个8位加法器级联
  - 4个4位加法器级联
- 32位加法器
  - … …

组合电路基本构件

- 多路复用器 Multiplexer (Mux)
  - 输入：$D_0, \ldots D_{n-1}$,
  - select
  - 输出：Y
  - 功能：在 N 个输入D_i中选择第2^{select}路输入连接到输出Y

2选1



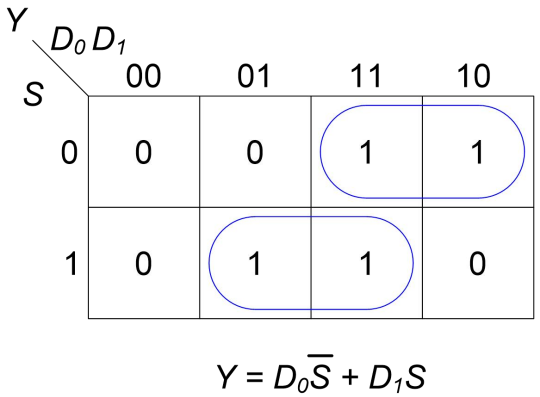
S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

S	Y
0	D_0
1	D_1

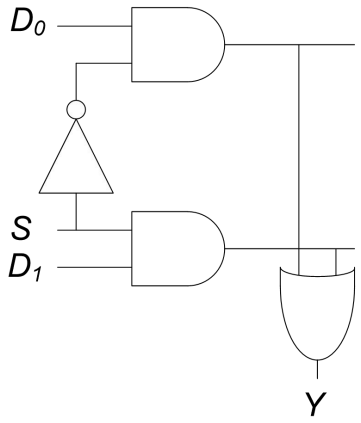
$$Y = D_0\overline{S} + D_1S$$

2选1开关：MUX 2：1实现

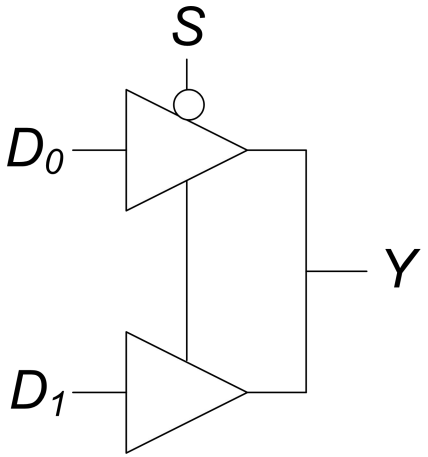
实现1：逻辑门



$$Y = D_0\overline{S} + D_1S$$

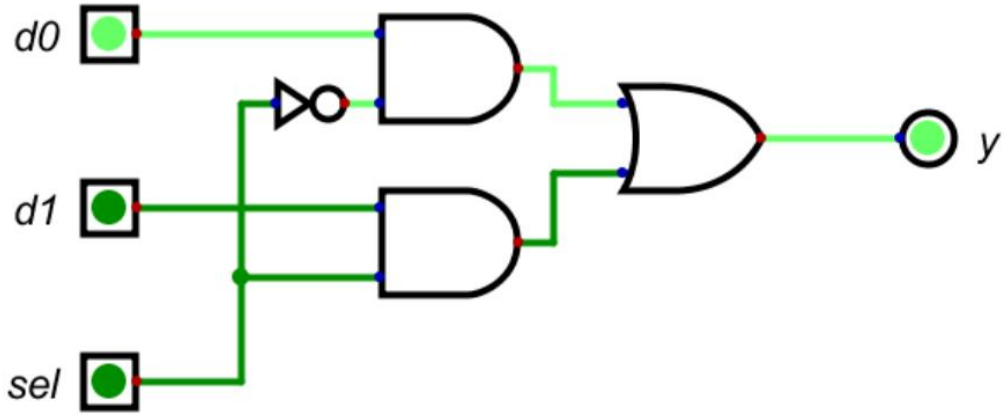


实现2：三态开关

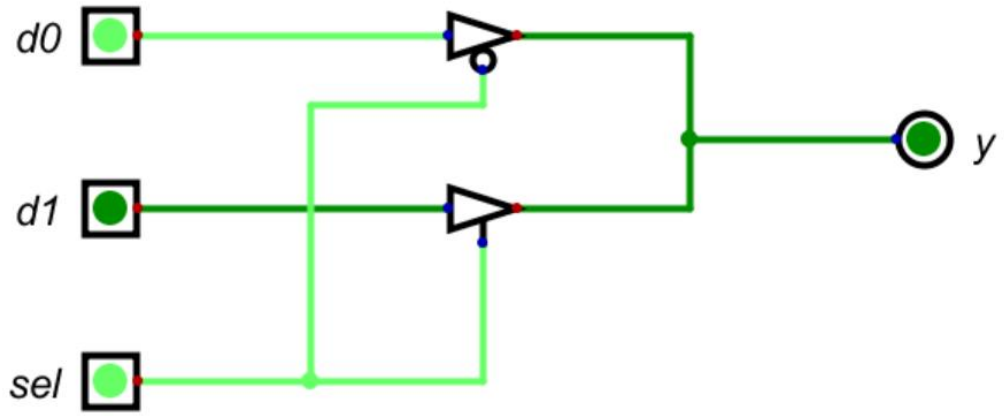


2选1开关：MUX 2：1实现

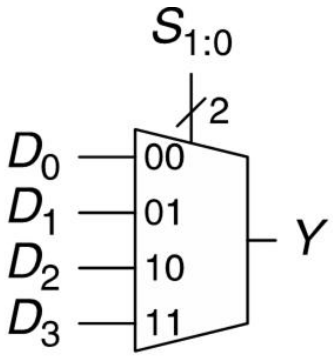
实现1：逻辑门



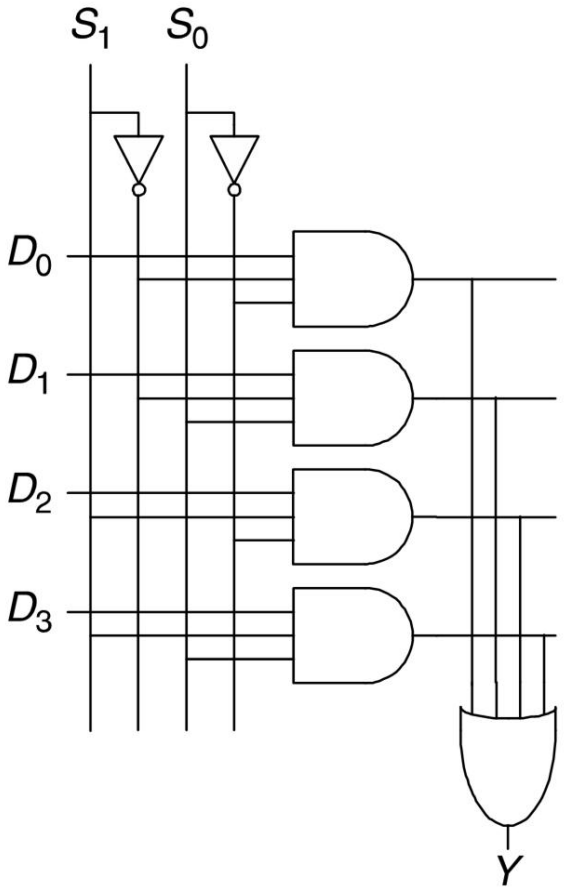
实现2：三态开关



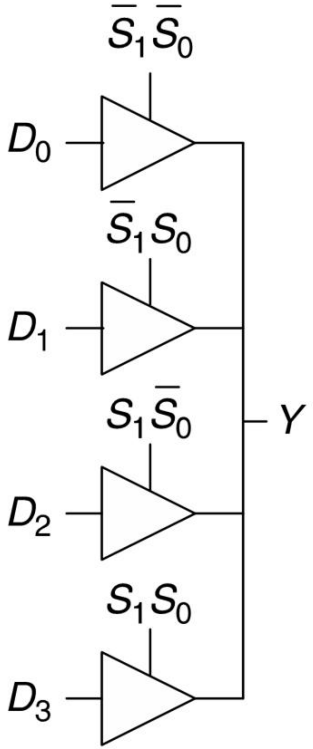
■ 4选1开关：MUX 4：1实现



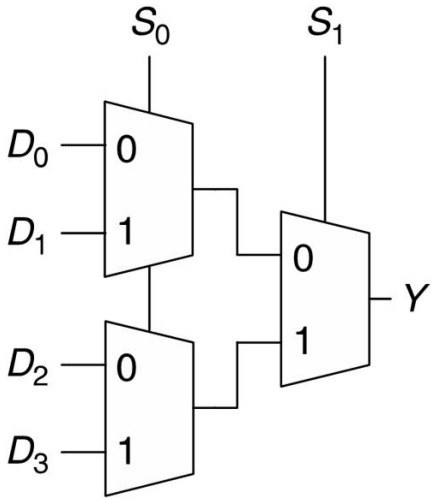
实现1：逻辑门



实现2：三态开关



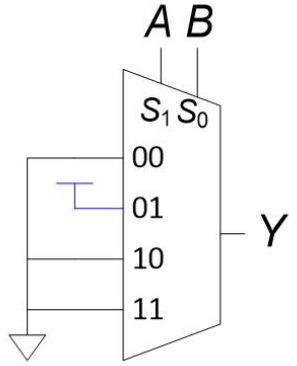
实现3：分层



用MUX实现逻辑表达式

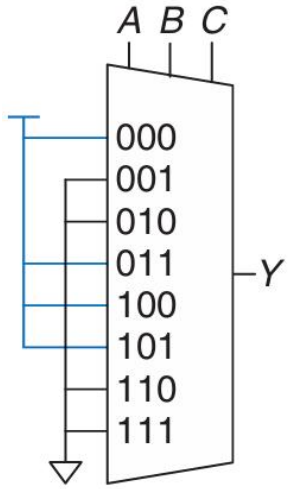
A	B	Y
0	0	0
0	1	1
1	0	0
1	1	0

$Y = \overline{A}B$



A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

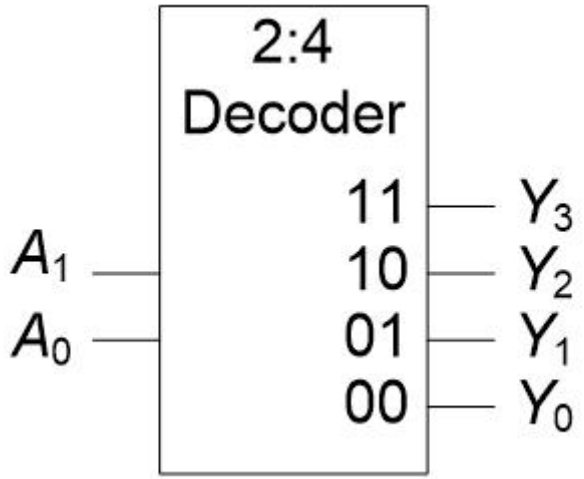
$Y = A\overline{B} + \overline{B}\overline{C} + \overline{A}BC$



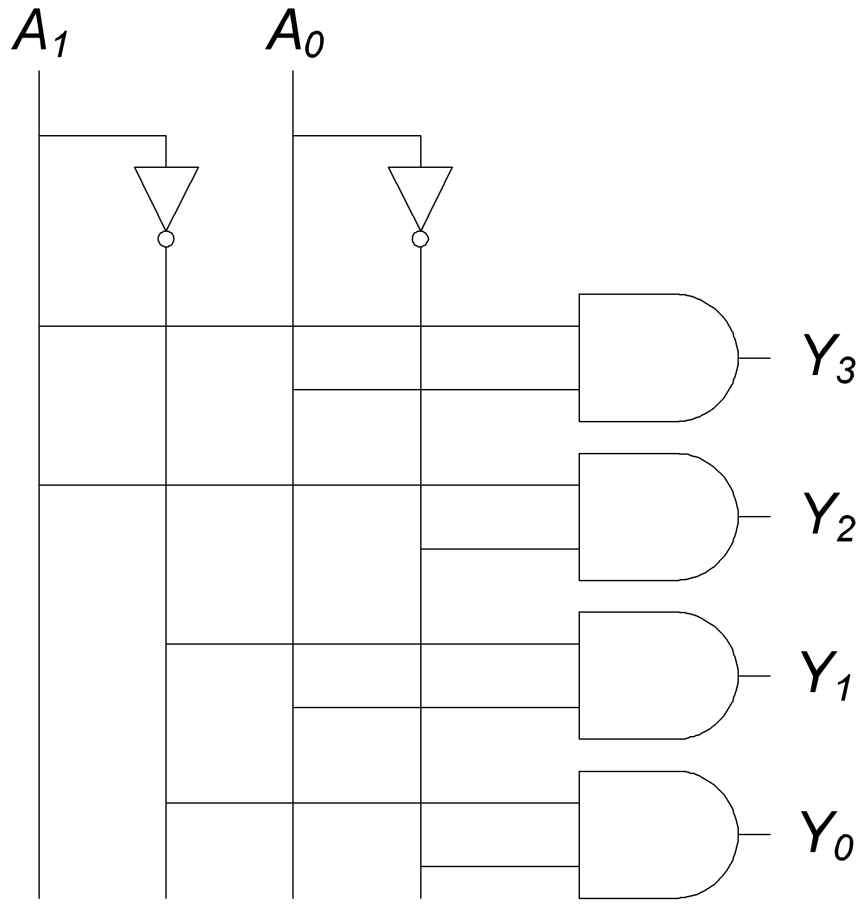
译码器(Decoder)

- 输入：A_0、A_1、…A_N
- 输出：2^N个输出
- 功能：编号为$A_N…A_1A_0$的输出为1，其余为0（one-shot，独热式）

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	**1**
0	1	0	0	**1**	0
1	0	0	**1**	0	0
1	1	**1**	0	0	0

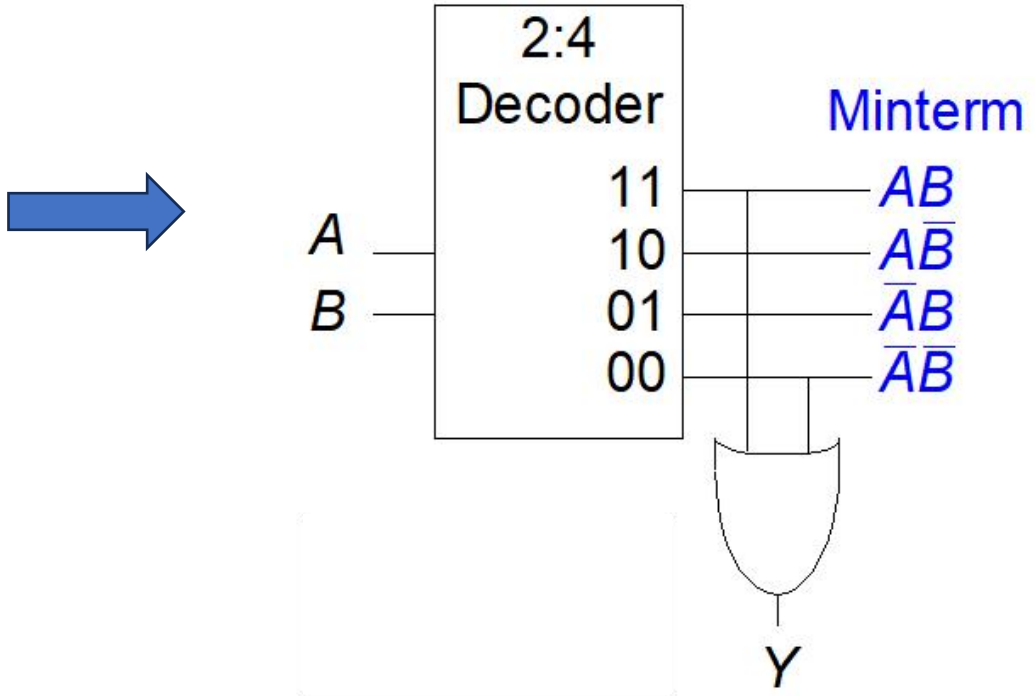


译码器(Decoder)实现

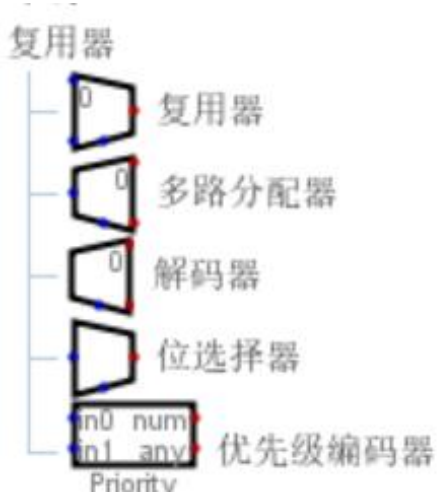


用译码器(Decoder)实现逻辑

$$Y = AB + \bar{A}\bar{B}$$
$$= \overline{A \oplus B}$$



Digital中的部件



硬件描述语言（HDL：Hardware Description Language）

- 硬件描述语言（Hardware Description Language, HDL）是一种用于描述电子系统硬件结构和行为的编程语言。它主要用于设计数字电路和系统。HDL允许工程师以文本形式描述电路的功能和结构，然后通过仿真和综合工具将其转换为实际的硬件实现。
  - 抽象层次：支持从行为级到门级的不同抽象层次描述。
  - 并行性：能够描述电路中并行操作，反映硬件并行特性。
  - 可综合性与可仿真性：既可用于仿真验证，也可通过综合工具生成实际硬件。

常用硬件描述语言

- VHDL：起源于美国国防部，语法严谨，适合复杂系统设计。
- VerilogHDL：语法类似C语言，易于学习，广泛应用于工业设计。
- SystemVerilog：Verilog的扩展，增加了系统级设计和验证功能。
- SystemC(C++)：基于 C++ 的硬件描述和系统级建模语言，特别适合复杂 SoC（系统级芯片）的设计和验证。
- Chisel(scala)：基于 Scala 的硬件描述语言，由 UC Berkeley 开发。它允许开发者使用 Scala 的高级特性来生成 Verilog 代码。
- SpinalHDL(scala)：基于 Scala 的硬件描述语言，旨在提供比 Chisel 更简洁和灵活的语法。
- Migen(Python)：Migen 是一个基于 Python 的硬件描述语言，旨在简化数字电路设计。它是 LiteX 项目的核心部分，广泛用于 FPGA 设计。

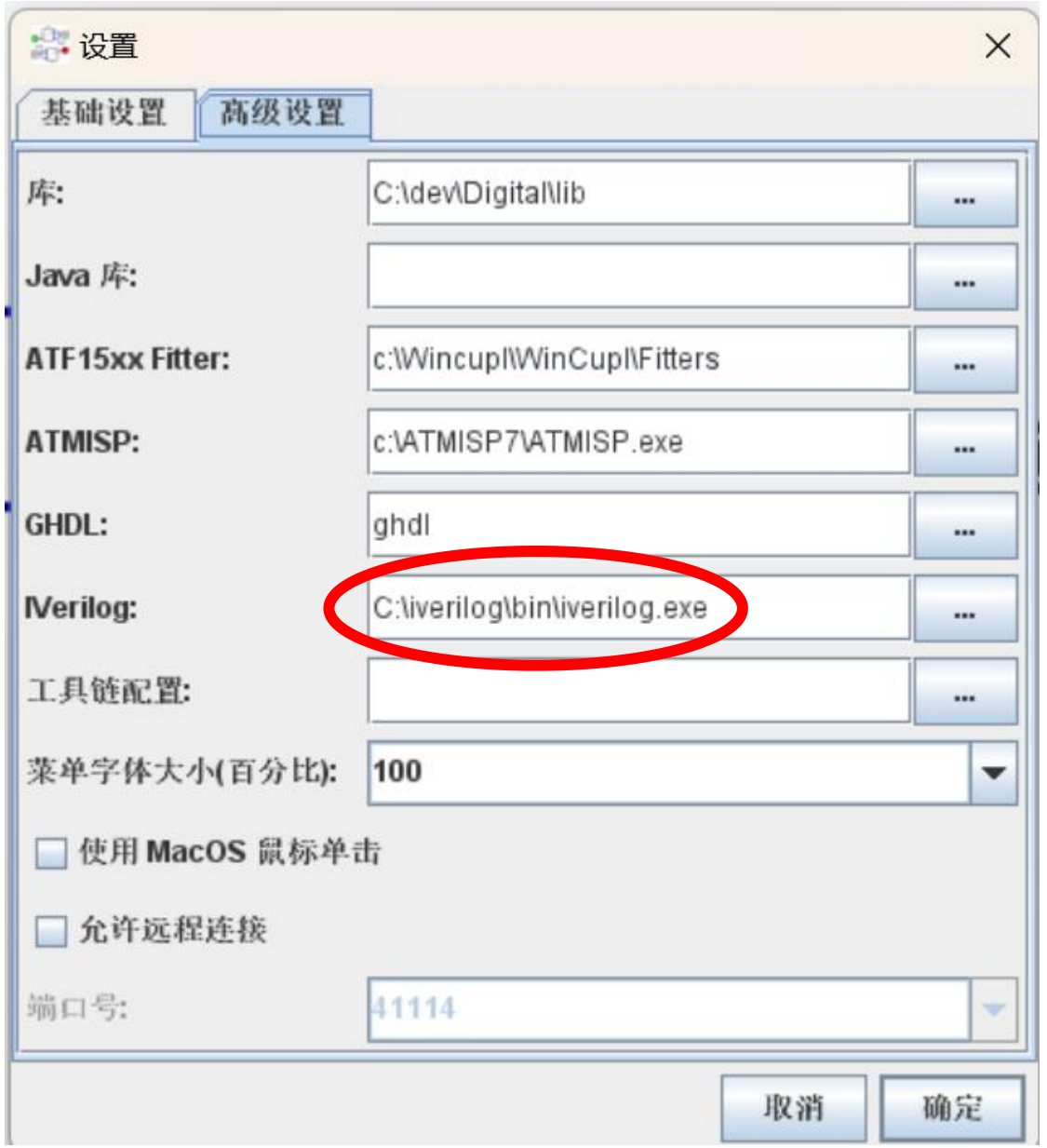
硬件描述语言

- 模拟
  - 应用于电路的输入
  - 检查输出的正确性
  - 通过模拟调试而不是硬件，节省开发费用
- 综合
  - 将HDL代码转换为描述硬件的网表（即逻辑门列表和连接它们的电线）

Digital中使用VerilogHDL

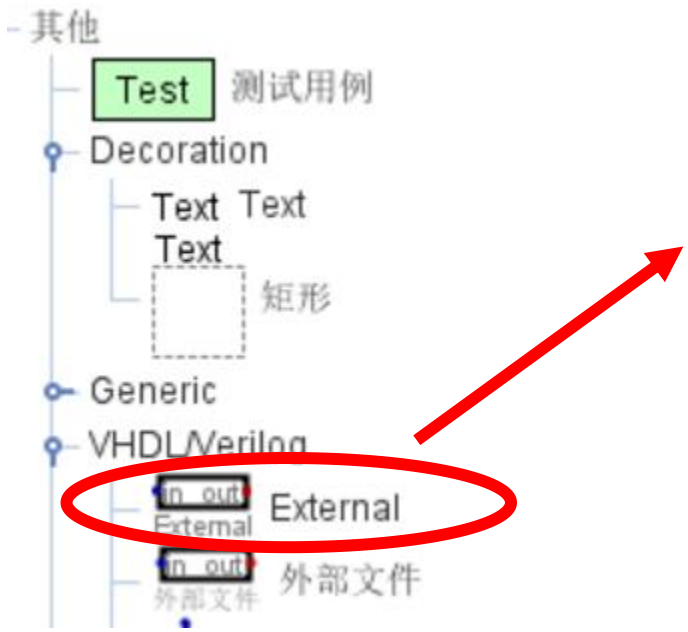
- Digital中是通过调用外部程序支持VerilogHDL和VHDL
  - VHDL：ghdl
  - VerilogHDL: Icarus Verilog (iverilog)

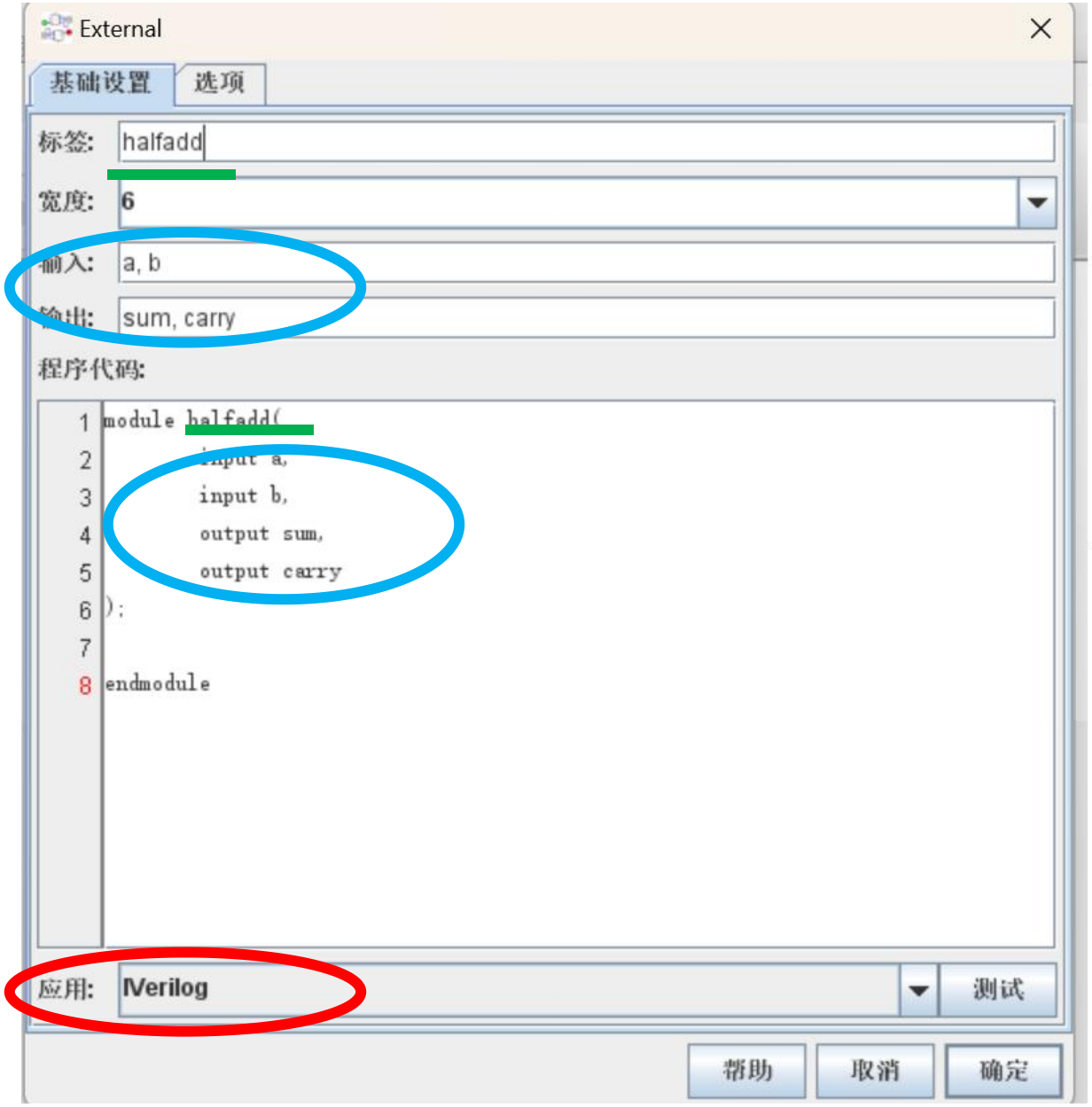
- iverilog安装（windows）
  - 下载：https://bleyer.org/icarus/
  - 建议安装在c:\iverilog目录下
  - 在Digital中打开”编辑->设置->高级设置”，选择iverilog.exe
  - 重启Digital



Digital中使用VerilogHDL

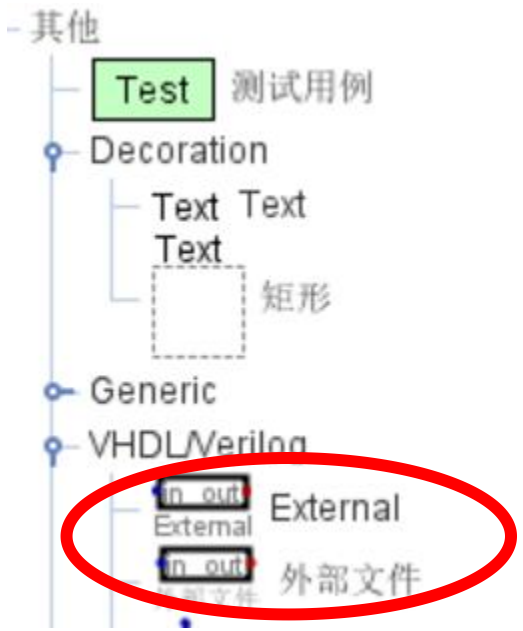
- Digital有两种方式使用verilogHDL
  - 在Digital中编写verilogHDL代码
  - 调用外部文件





Digital中使用VerilogHDL

- Digital有两种方式使用verilogHDL
  - 在Digital中编写verilogHDL代码
  - 调用外部文件



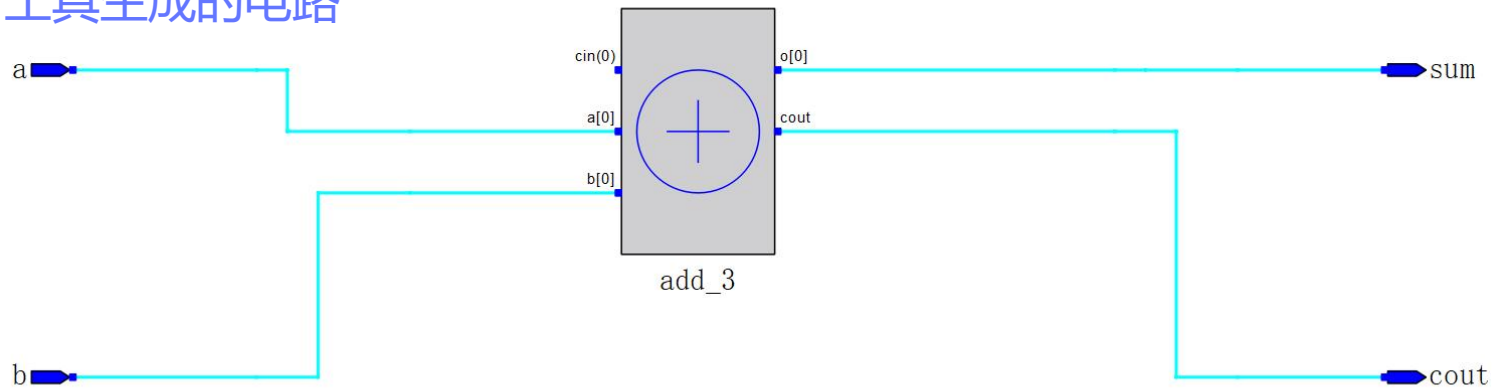
1位半加器（HalfAdder）

- 输入：a, b
- 输出：sum, carry
- 功能描述：
- 　　{ carry, sum } = a + b

行为级描述

```
module halfadd(
    input a,
    input b,
    output sum,
    output carry
);

    assign {carry, sum} = a + b;

endmodule
```

- Gowin FPGA综合工具生成的电路



1位半加器（HalfAdder）

- 输入：a, b
- 输出：sum, carry
- 功能描述：
- { carry, sum } = a + b

a	b	sum	carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

```
module halfadd(
    input a,
    input b,
    output sum,
    output carry
);
    assign {carry, sum} =
                {a, b} == 0? 2'b00 :
                {a, b} == 1? 2'b01 :
                {a, b} == 2? 2'b01 :
                {a, b} == 3? 2'b10 : 0;


endmodule
```

真值表

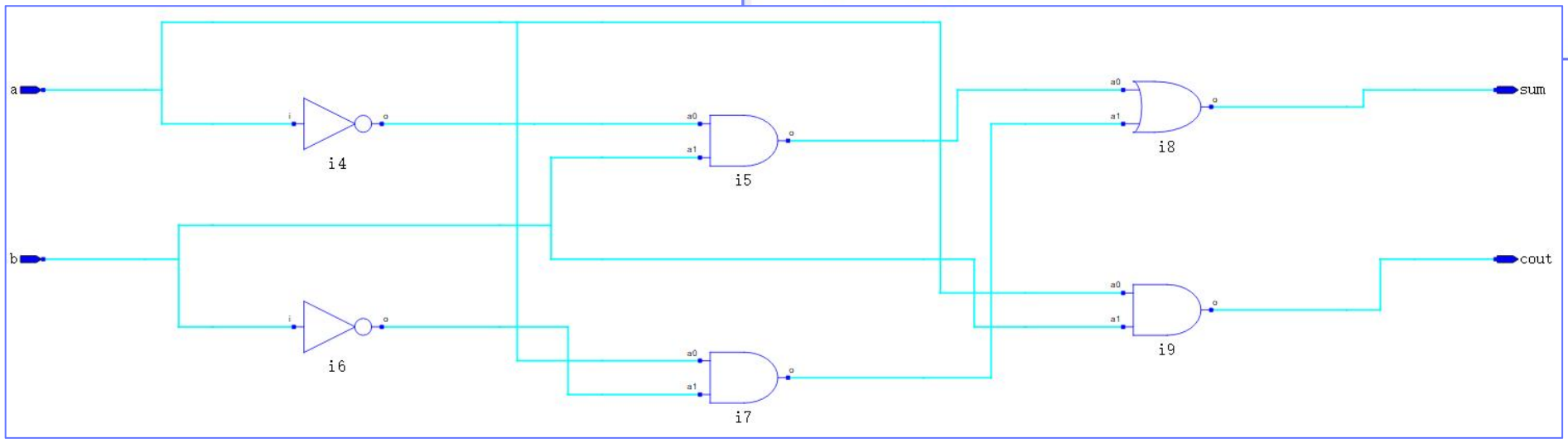
1位半加器（HalfAdder）

- 输入：a, b
- 输出：sum, carry
- 功能描述：
- { carry, sum } = a + b

sum = (~a & b) | (a & ~b)
carry = a & b

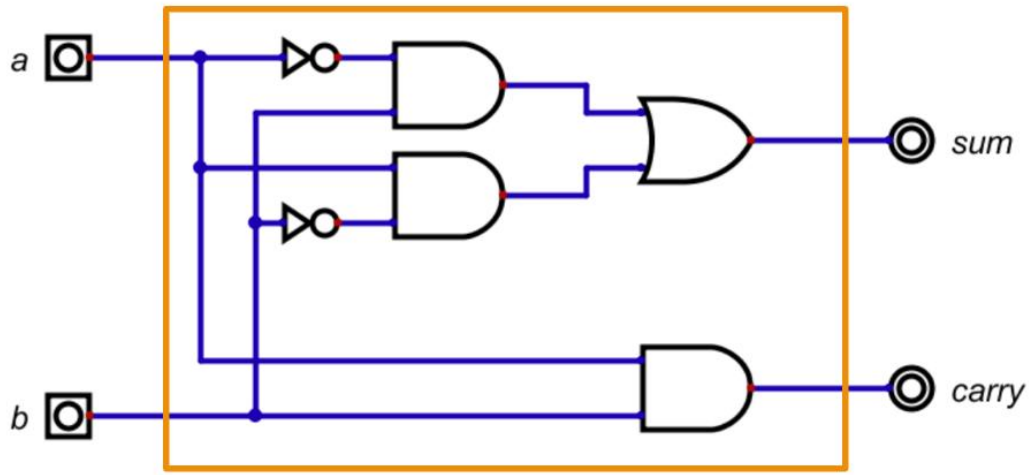
```
module halfadd(
    input a,
    input b,
    output sum,
    output carry
);
    assign sum = (~a & b) | (a & ~b);
    assign carry = a & b;
```

布尔表达式



1位半加器（HalfAdder）

- 输入：a, b
- 输出：sum, carry
- 功能描述：
- { carry, sum } = a + b



门级描述

```
module halfadd(
    input a,
    input b,
    output sum,
    output carry
);
    wire a_, b_, a1_out, a2_out;

    not n1(a_, a);
    not n2(b_, b);

    and a1(a1_out, a_, b);
    and a2(a2_out, a, b_);

    or o1(sum, a1_out, a2_out);

    and a3(carry, a, b);

endmodule
```

全加器的verilog实现

- 输入：a, b, cin
- 输出：sum, cout
- 功能描述：
- { cout, sum } = a + b + cin

```
module add1(
    input a,
    input b,
    input cin,
    output sum,
    output cout
);

    assign {cout, sum} = a + b + cin;
endmodule
```

```
module add1(
    input a,
    input b,
    input cin,
    output sum,
    output cout
);

    halfadd h1(a,b,s1, c1);
    halfadd h2(s1, cin, sum, c2);

    or o1(cout, c1, c2);
endmodule
```

多位加法器

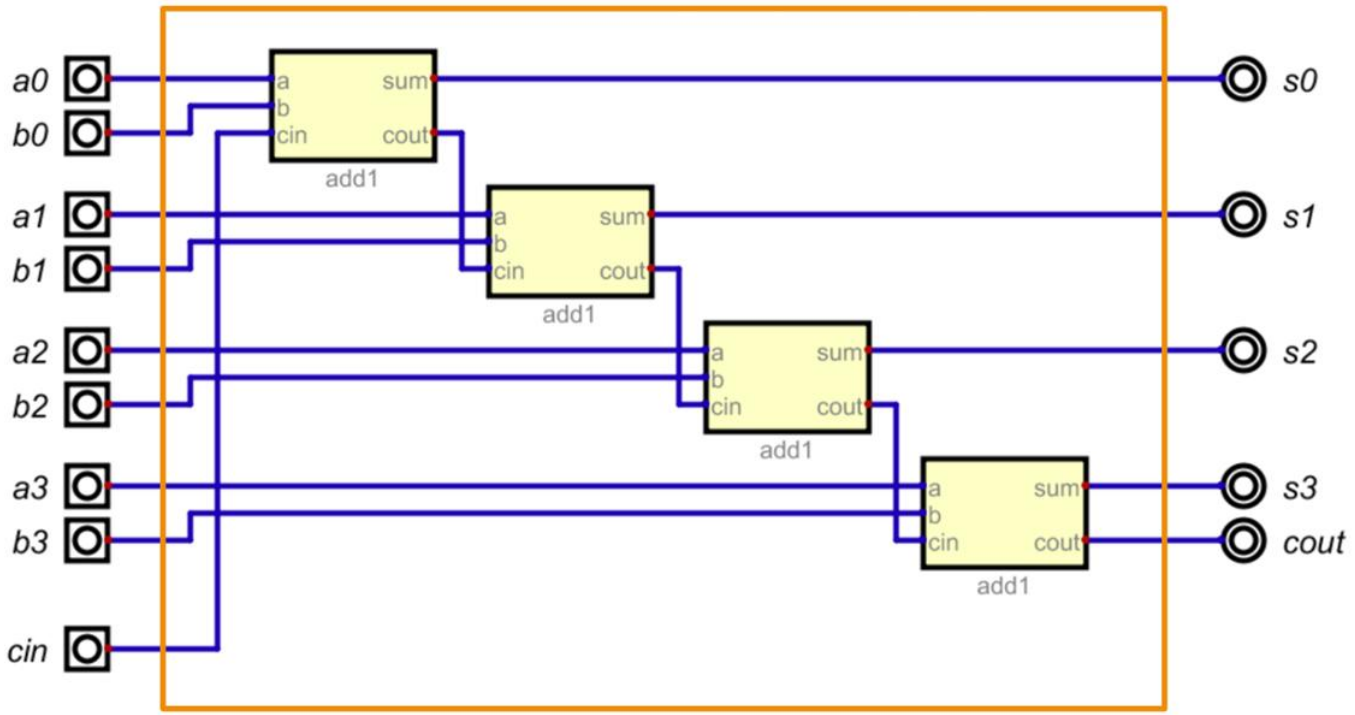
```
module add4(
    input [3:0] a,
    input [3:0] b,
    input cin,
    output [3:0] sum,
    output cout
);

    wire c1, c2, c3;

    add1 a0(a[0], b[0], cin, sum[0], c1);
    add1 a1(a[1], b[1], c1, sum[1], c2);
    add1 a2(a[2], b[2], c2, sum[2], c3);
    add1 a3(a[3], b[3], c3, sum[3], cout);

endmodule
```

```
module add4(
    input [3:0] a,
    input [3:0] b,
    input cin,
    output [3:0] sum,
    output cout
);

    assign {cout, sum} = a + b + cin;
endmodule
```

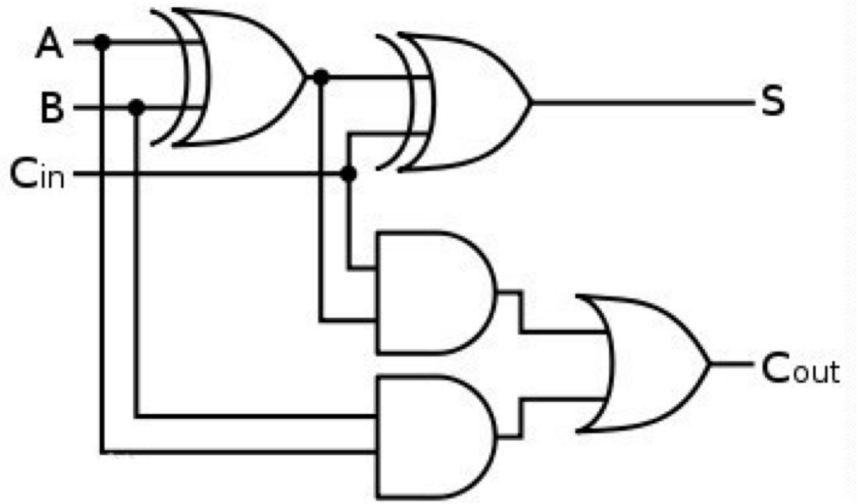
超前进位加法器(CLA: Carry-Lookahead Adder)

- 串行进位加法器
  - 速度慢：进位输出延迟大

- 超前进位加法器
  - 改进：直接计算出进位
  - 空间换时间



超前进位加法器



$$C_{i+1} = (A_i \cdot B_i) + (A_i \cdot C_i) + (B_i \cdot C_i)$$
$$= (A_i \cdot B_i) + (A_i + B_i) \cdot C_i$$

设：

- 生成（Generate）信号：$G_i = A_i \cdot B_i$
- 传播（Propagate）信号：$P_i = A_i + B_i$

则：$C_{i+1} = G_i + P_i \cdot C_i$

超前进位加法器

- $C_1 = G_0 + P_0 \cdot C_0$
- $\mathbf{C_2} = G_1 + P_1 \cdot C_1$

  $= G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0)$

  $\mathbf{= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0}$
- $\mathbf{C_3} = G_2 + P_2 \cdot C_2$

  $= G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0)$

  $\mathbf{= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0}$
- $\mathbf{C_4} = G_3 + P_3 \cdot C_3$

  $= G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0)$

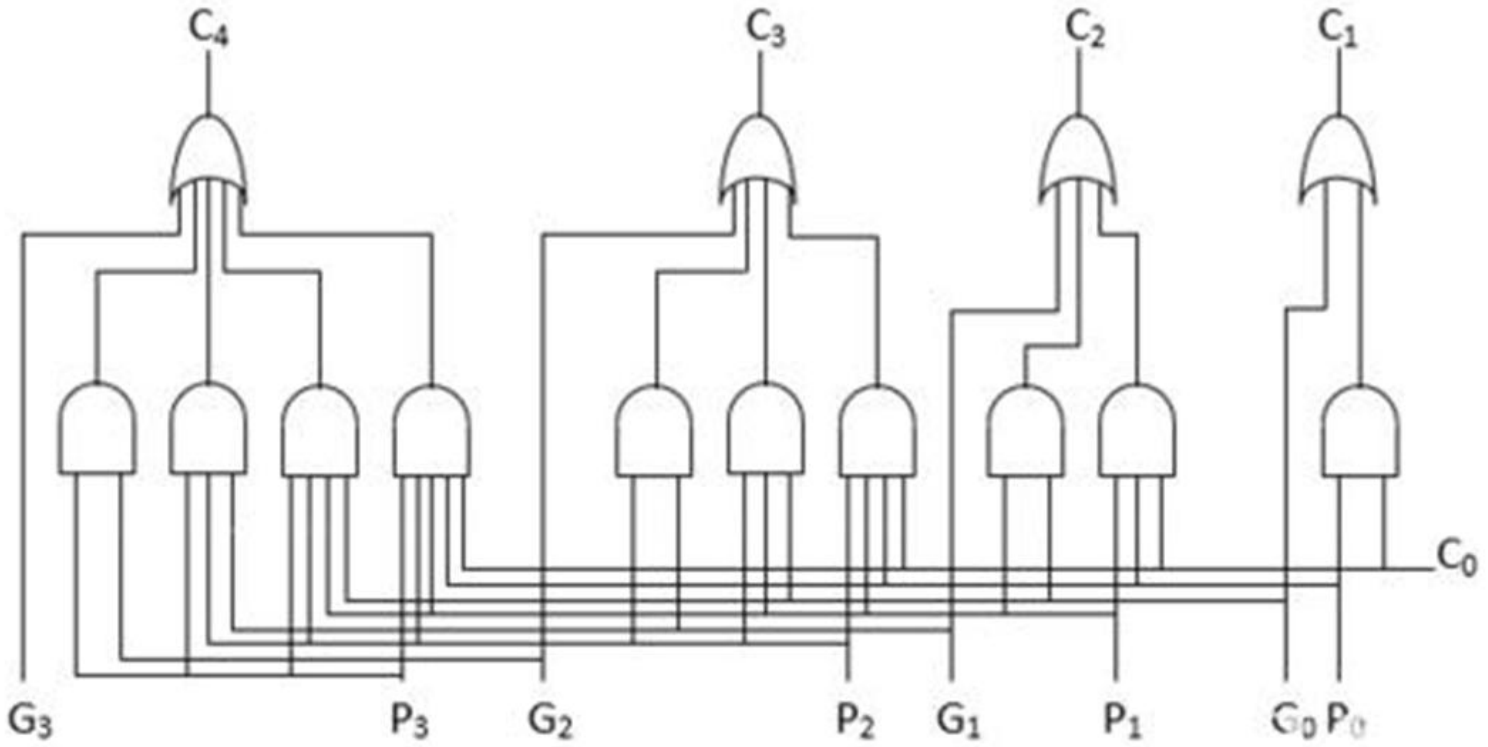
  $\mathbf{= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0}$

$G_i = A_i \cdot B_i$

$P_i = A_i + B_i$

$\mathbf{C_{i+1} = G_i + P_i \cdot C_i}$

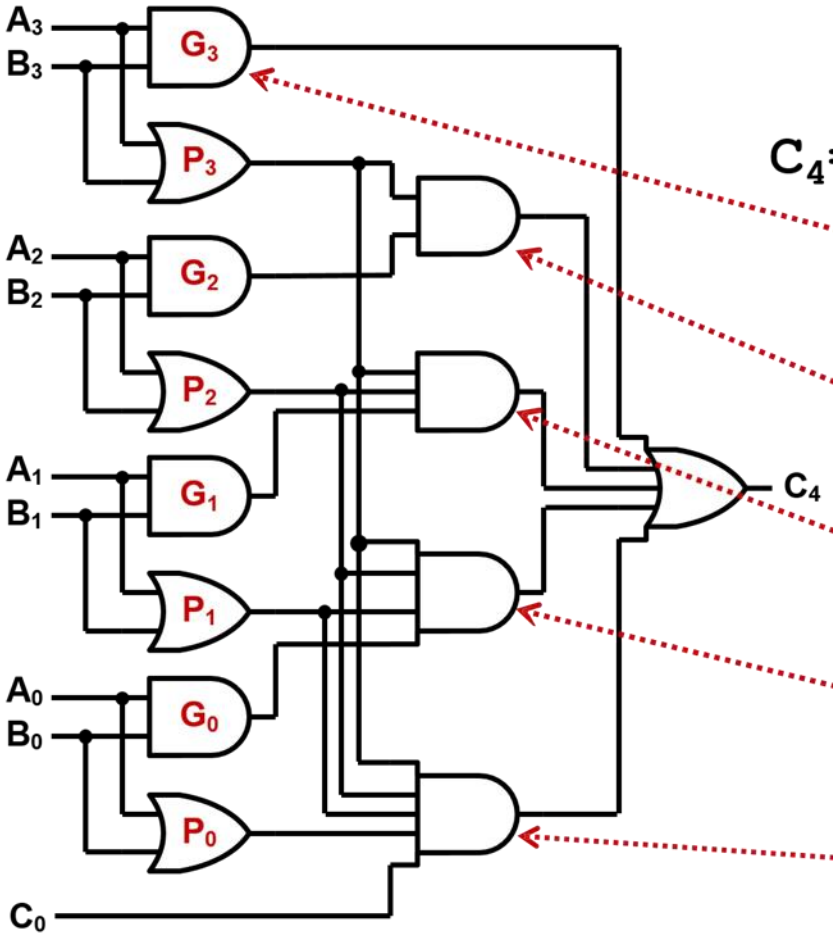
超前进位加法器



$$G_i = A_i \cdot B_i$$
$$P_i = A_i + B_i$$

超前进位加法器：关键路径

$$G_i = A_i \cdot B_i$$
$$P_i = A_i + B_i$$

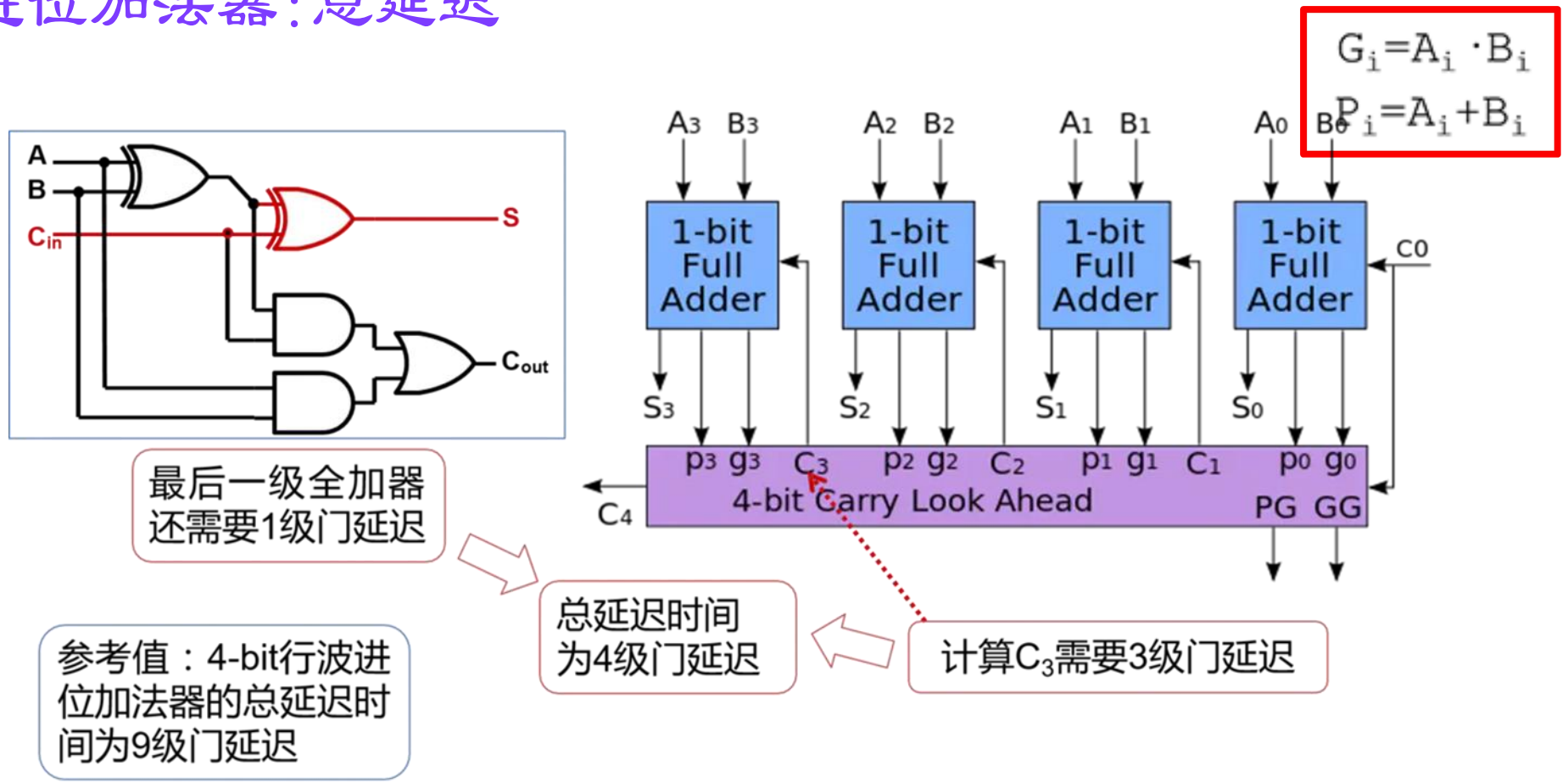


$$C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

优点：计算C_{i+1}的延迟时间固定为三级门延迟，与 加法器的位数无关

缺点：如果进一步拓宽加法器的位数，则电路变得非常复杂

超前进位加法器：总延迟



如果是64bit加法器，采用行波进位加法器需要129级门延迟；如果采用超前进位，理论上仍是4级门延迟。

时序逻辑

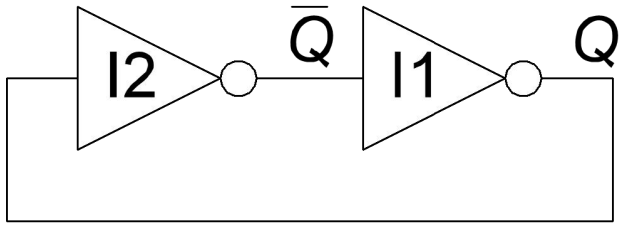
- 有记忆（状态）
- 输出不仅与当前输入有关，还与过去的输入有关

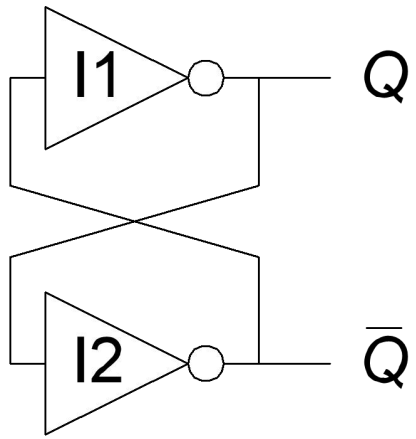
状态器件

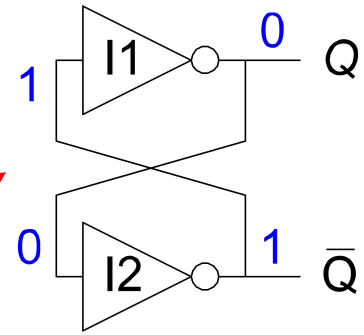
- 状态
  - 预测电路未来行为所需的所有先前输入
- 状态器件
  - 用于存储状态
    - 双稳态电路
    - SR锁存器
    - D锁存器
    - D触发器

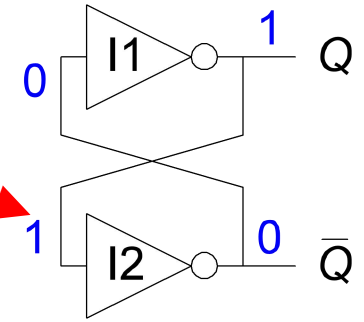
双稳态电路

- 状态器件的基础构件
- 有两个输出：Q、$\overline{Q}$
- 没有输入
- 反馈



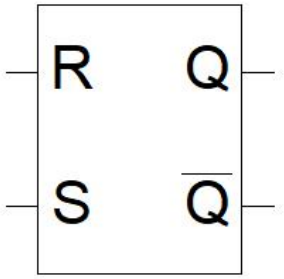


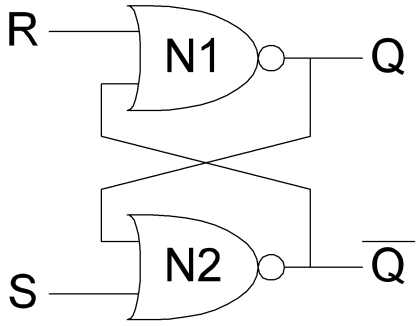


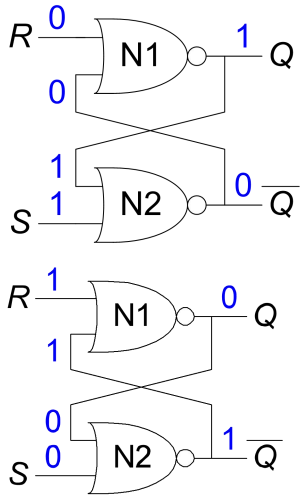


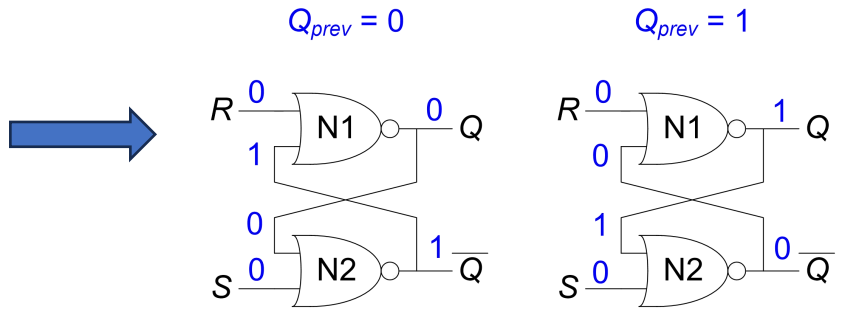
SR锁存器(保存1位状态)

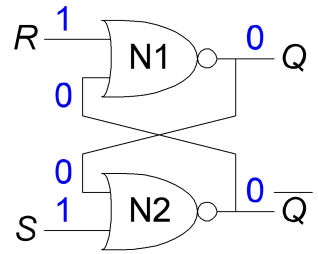
- 输入：R (reset, 复位) 、S（set, 置位)
- 输出：Q、$\overline{Q}$
- 功能:
  - Set使输出为1： S = 1, R = 0, Q = 1
  - Reset 使输出为0: S = 0, R = 1, Q = 0
  - 保持输出不变: S = 0, R = 0, Q = $Q_{前值}$
  - 非法: S = 1, R = 1





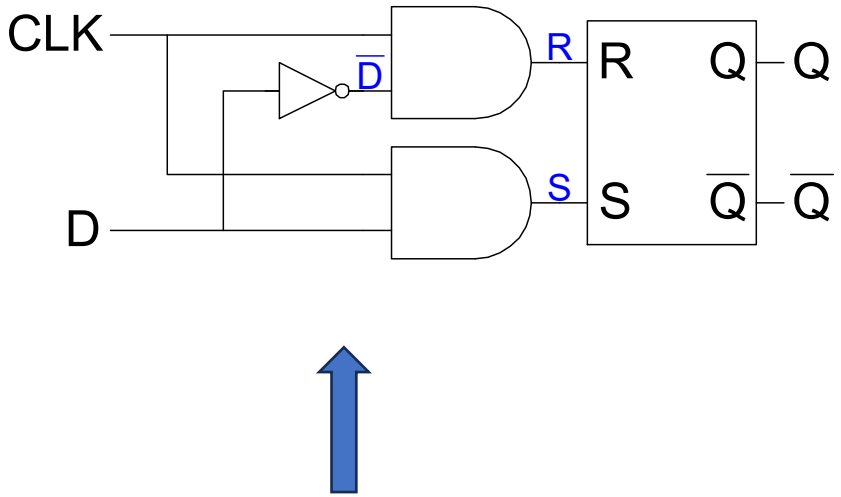


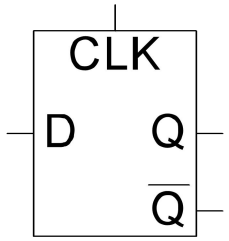




D锁存器(保存1位状态)

- 输入：CLK、D
- 输出：Q、$\overline{Q}$
- 功能：
- CLK为0时，输出不变
- CLK为1时，输出Q为D



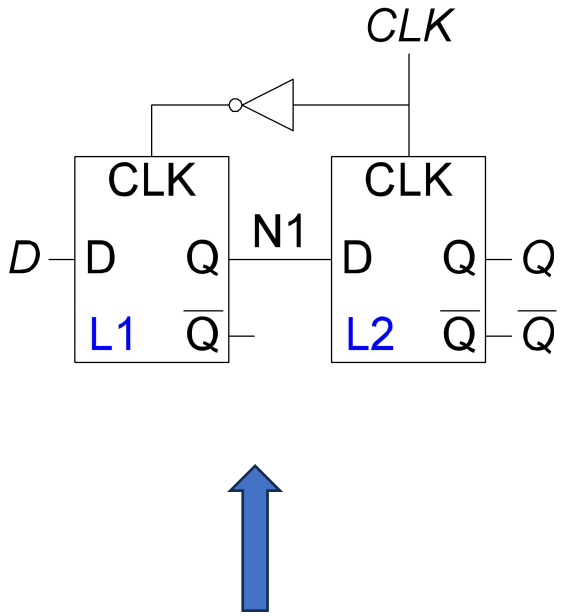


CLK	D	$\overline{D}$	S	R	Q	$\overline{Q}$
0	X	$\overline{X}$	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

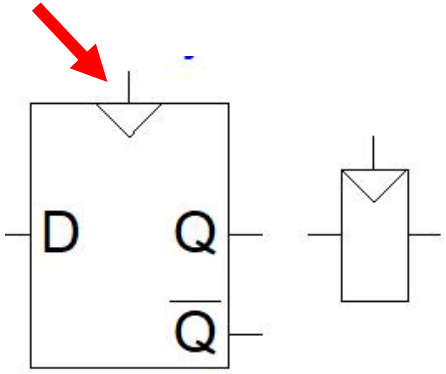
```
module d_latch(
    input clk,
    input d,
    output reg q
);

    always @(*)
    begin
        if(clk)
            q = d;
    end

endmodule
```

D触发器（同步电路）

- 输入：CLK、D
- 输出：Q、$\overline{Q}$
- 功能：
- CLK从0变为1时，输出为D（上升沿触发）
- 其余，输出不变

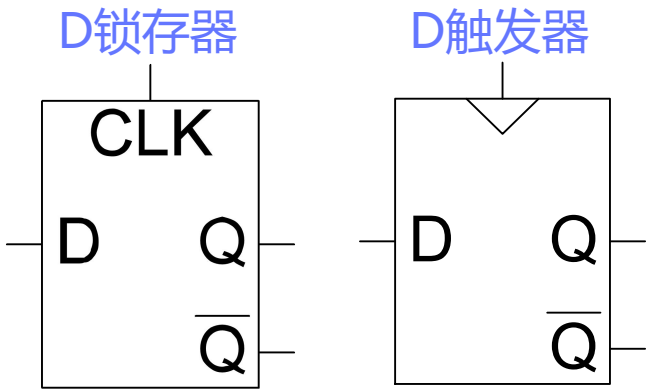


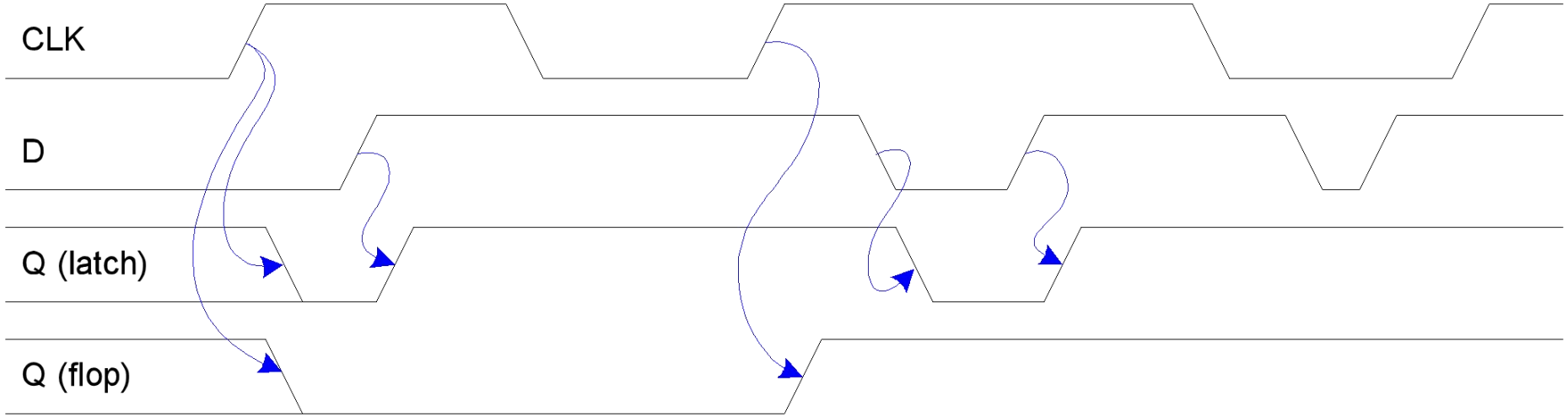

```
module dff(
    input clk,
    input d,
    output reg q
);

    always @(posedge clk)
    begin
        q = d;
    end

endmodule
```



- CLK为0时，D写入L1，　　L2输出不变
- CLK为1时，L1的Q写入L2，L1输出不变
- 在CLK的上升沿（clk从0到1），D写入Q

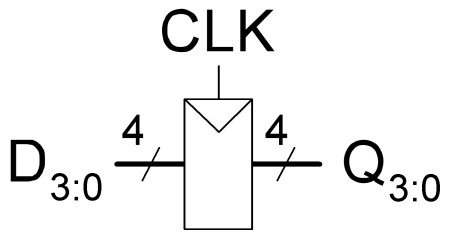
D锁存器和D触发器



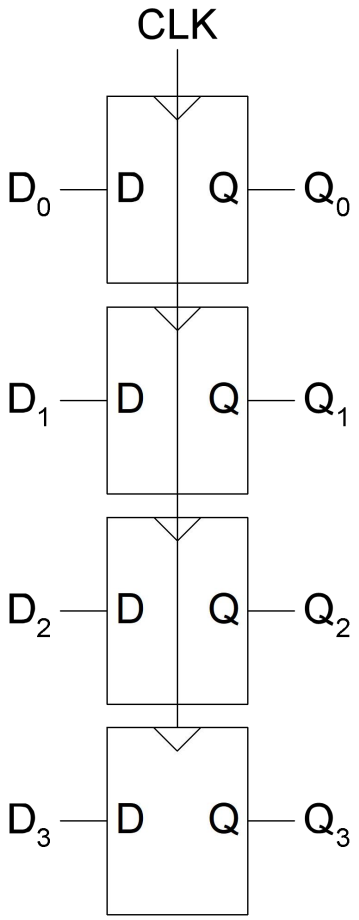


D触发器的变种

- 寄存器：多位D触发器

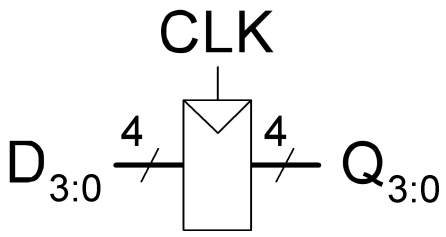


- 4位寄存器

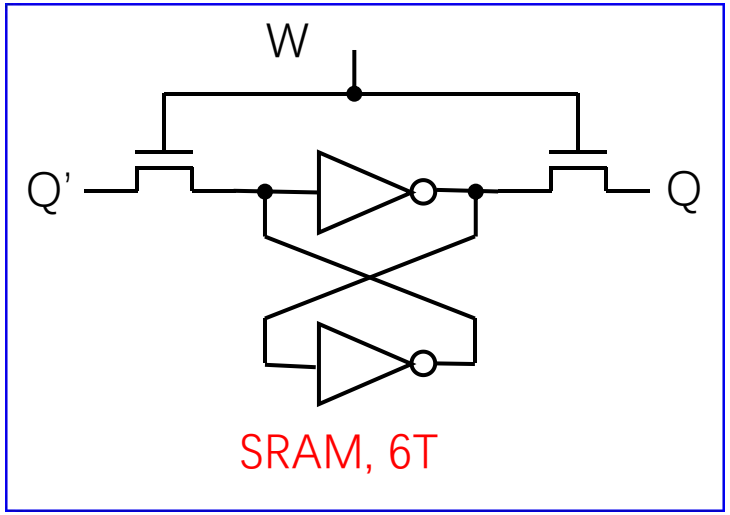


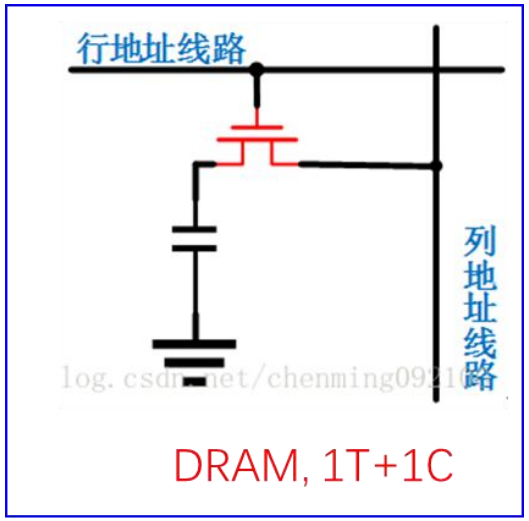

```
module reg4(
    input clk,
    input [3:0] d,
    output reg [3:0] q
);

    always @(posedge clk)
    begin
        q = d;
    end

endmodule
```

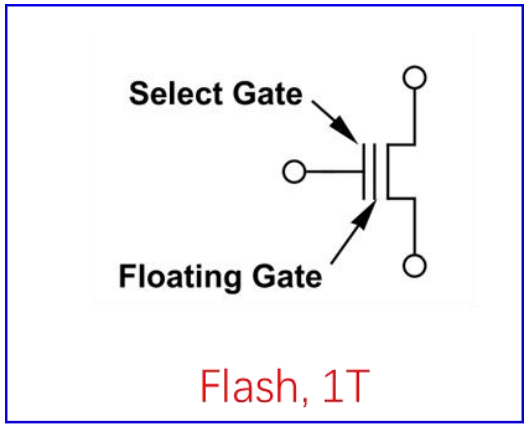
寄存器



- 4位寄存器
- 46个晶体管

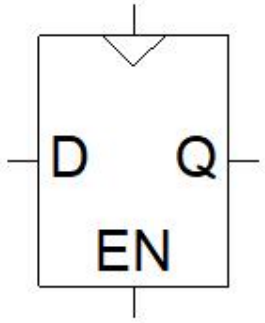


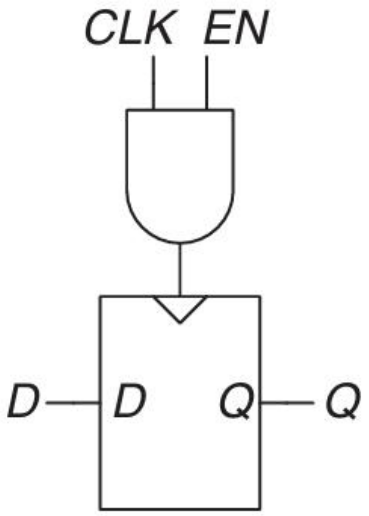


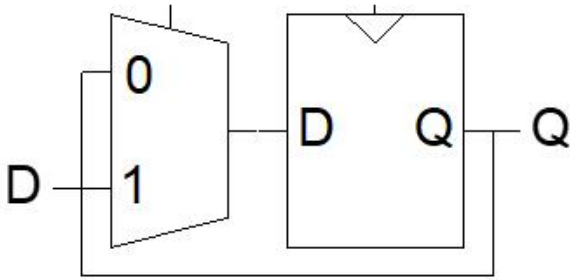


D触发器的变种

- 带使能en的D触发器
  - enable=1：在clk上升沿，Q=D；
  - enable=0：Q保持不变



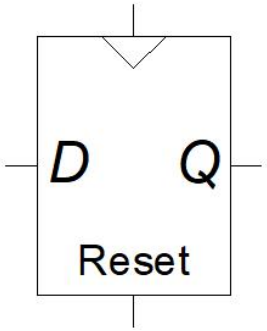


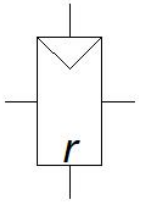



```
mdoule dff(
    input clk,
    input enble,
    input d,
    output reg q
);
    always @(posedge clk)
    begin
        if(enable)
            q = d;
    end
endmodule
```

D触发器的变种

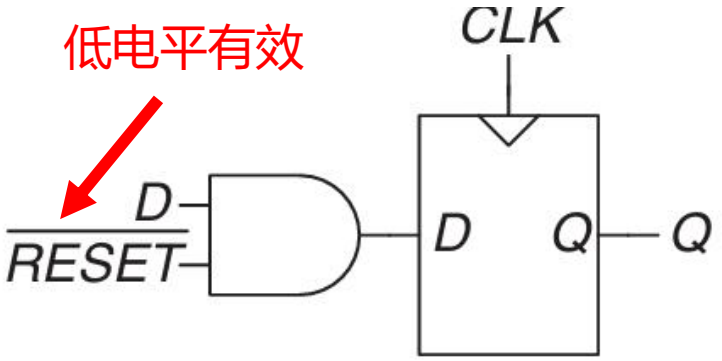
- 带复位的D触发器
  - reset=1：Q=0；
  - reset=0：同D触发器





D触发器的变种

- 带复位的D触发器
  - 同步复位
    - 复位信号（）在时钟的上升沿起作用
  - 异步复位

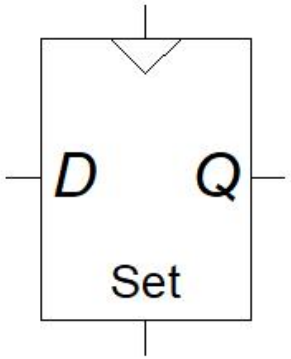


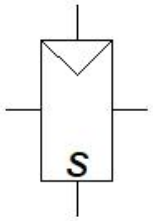

```
mdoule dff(
    input clk,
    input rst,
    input d,
    output reg q
);
    always @(posedge clk)
    begin
        if(rst)
            q = 0;
        else
            q = d;
    end
endmodule
```

```
mdoule dff(
    input clk,
    input rst,
    input d,
    output reg q
);
    always @(negedge rst or posedge clk)
    begin
        if(!rst)
            q = 0;
        else
            q = d;
    end
endmodule
```

D触发器的变种

- 带置位的D触发器
  - 在时钟的上升沿
    - 如果set=1，则q=1

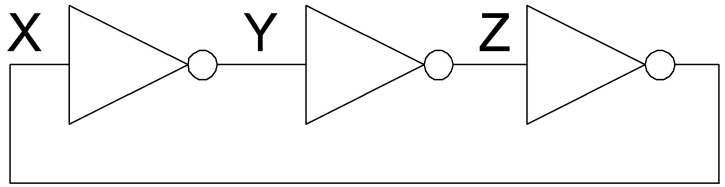


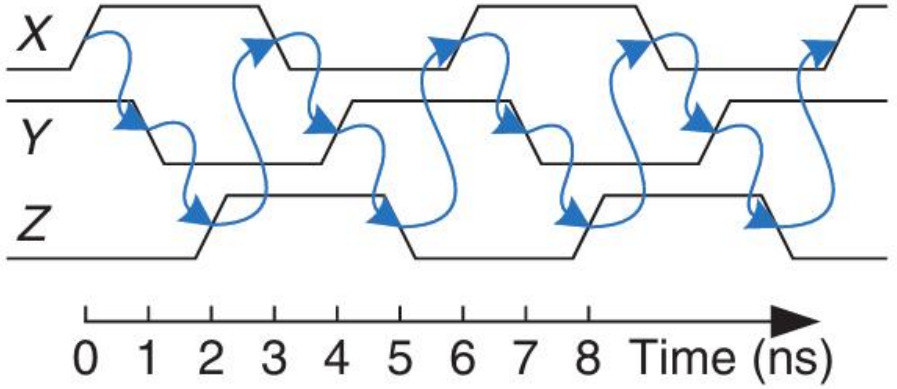



```
module dff(
    input clk,
    input set,
    input d,
    output reg q
);
    always @(posedge clk)
    begin
        if(set)
            q = 1;
        else
            q = d;
    end
endmodule
```

时序逻辑

- 不是所有的电路都是组合电路
- 时序电路的问题
  - 输出接到输入，形成回路



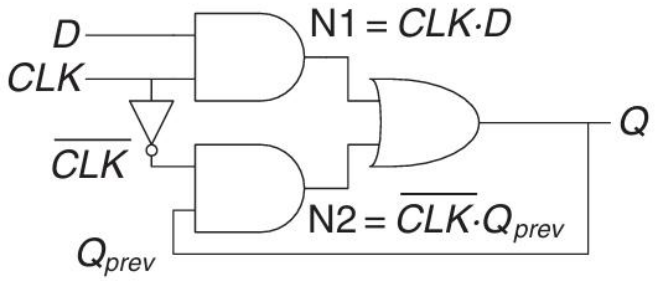


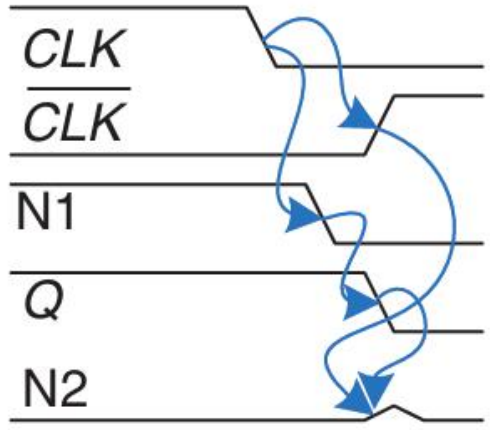
时序逻辑

- 不是所有的电路都是组合电路
- 时序电路的问题
  - 输出接到输入，形成回路

CLK	D	Q_{prev}	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$Q = CLK \cdot D + \overline{CLK} \cdot Q_{prev}$$





假定反相器的延迟比AND 和 OR 门的延迟大，节点 N1 和 Q 可能都在 $\overline{CLK}$ 上升之前下降。在这种情况下，N2 永远不会上升为1，而Q 始终为 0。

同步时序逻辑

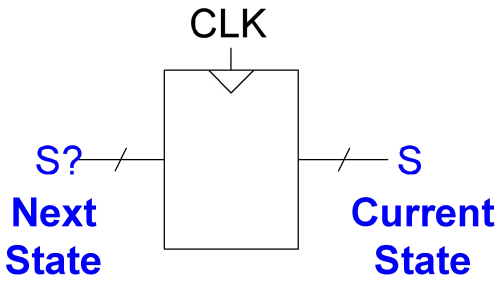
- 通过插入寄存器打破可能的回路
- 寄存器包含系统的状态
- 状态在时钟边缘变化
  - 系统通过时钟同步
- 同步时序电路组成规则：
  - 每个电路元件不是寄存器就是组合电路
  - 至少一个电路元件是寄存器
  - 所有寄存器使用相同的时钟
  - 每个回路至少包含一个寄存器

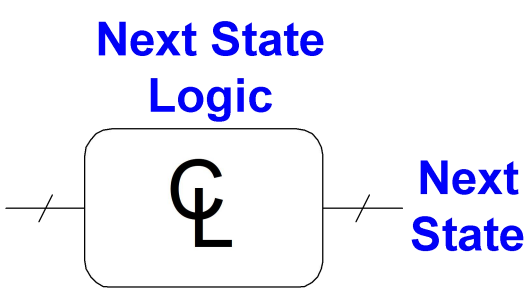
两种同步时序电路

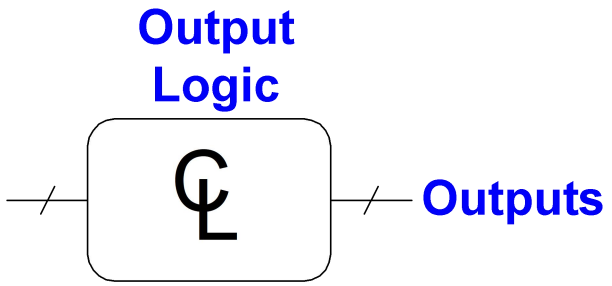
- 有限状态机（FSM：Finite State Machine）
- 流水线（Pipeline）

有限状态机

- 包括:
  - 状态寄存器
    - 存储当前状态
    - 在每个时钟的上升沿装入下一状态
  - 组合电路
    - 计算下一状态
    - 计算输出



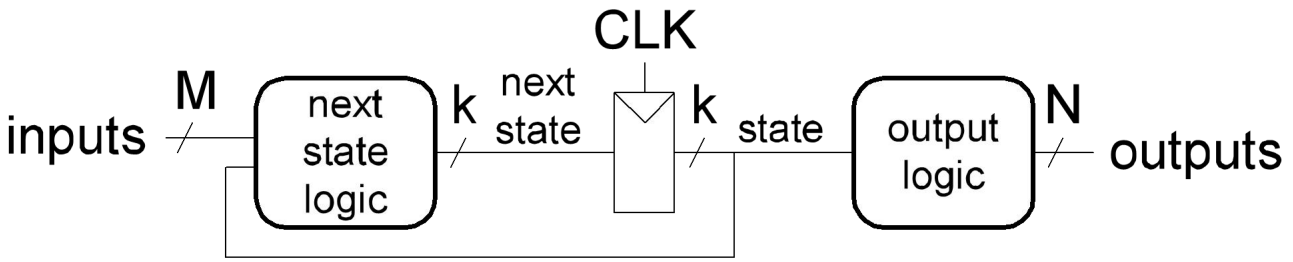




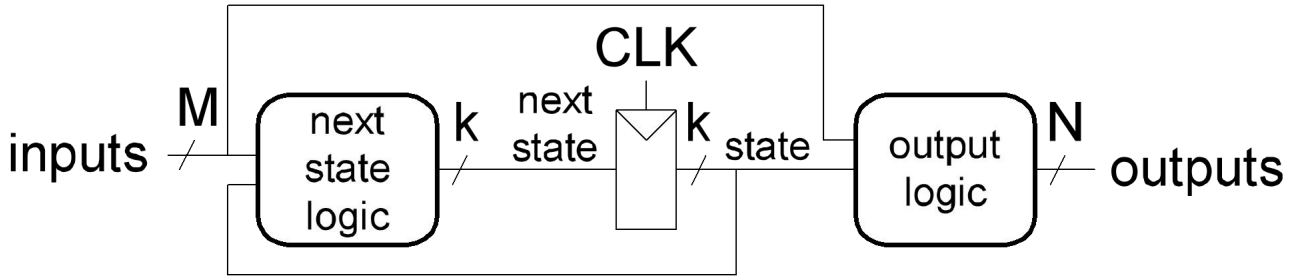
有限状态机

- 下一状态由当前状态和输入决定
- 输出
  - 摩尔型FSM
    - 输出仅取决于当前状态
  - 米利型FSM
    - 输出取决于当前状态和输入

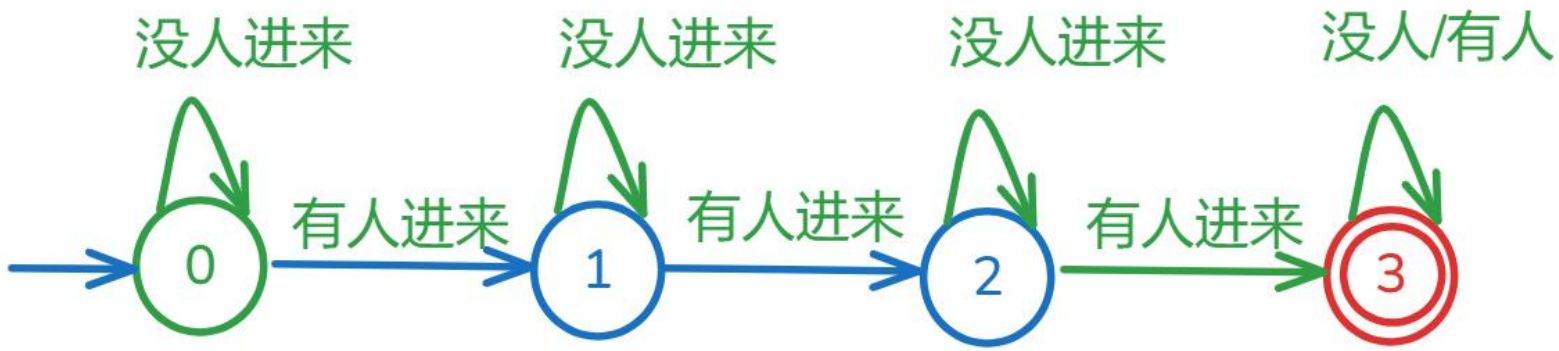
Moore FSM



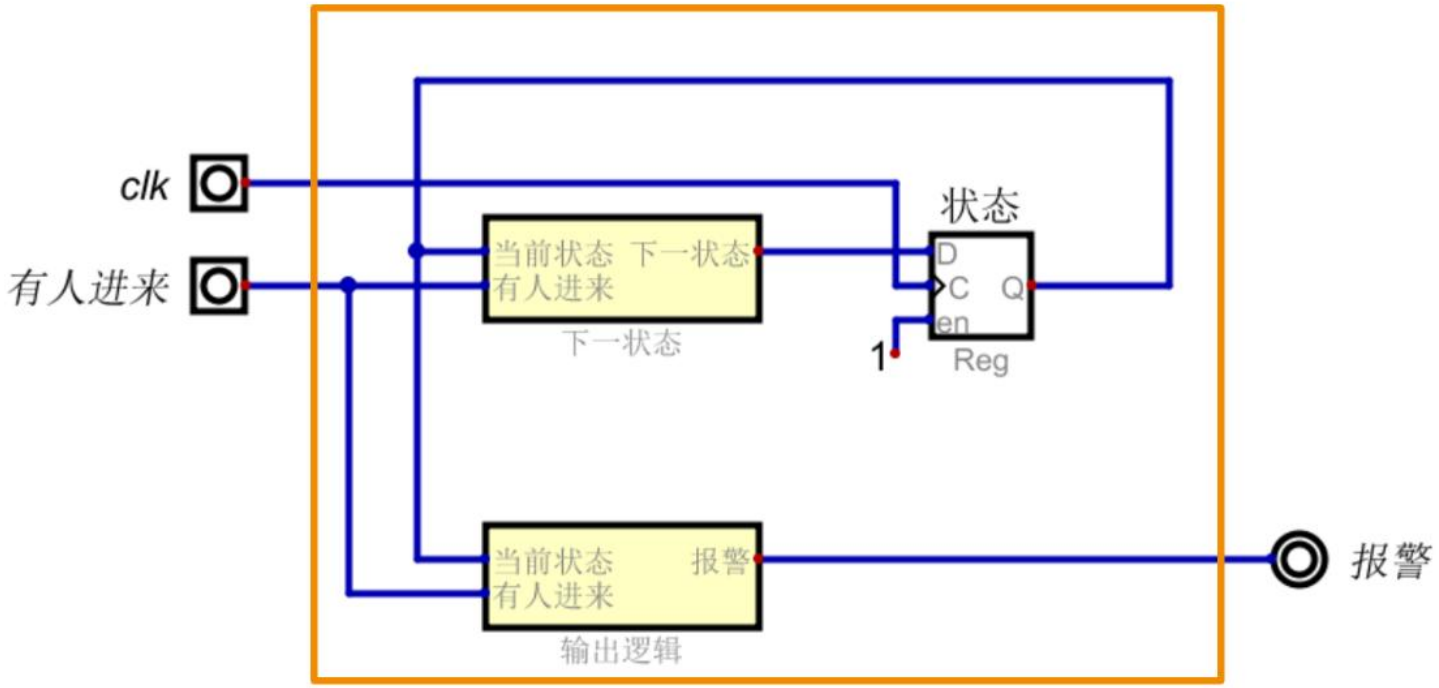
Mealy FSM



时序逻辑示例1

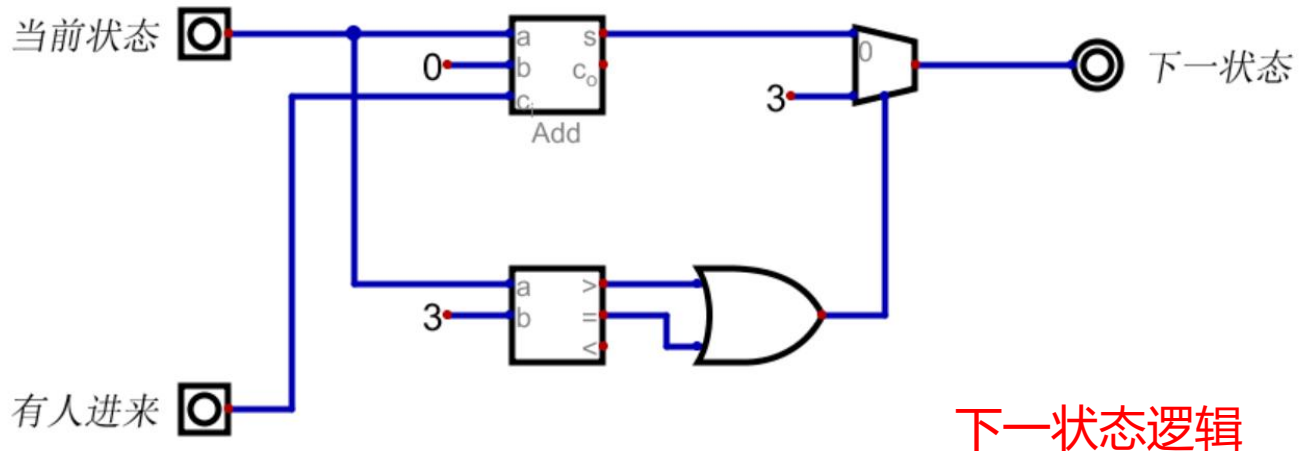


时序逻辑示例1

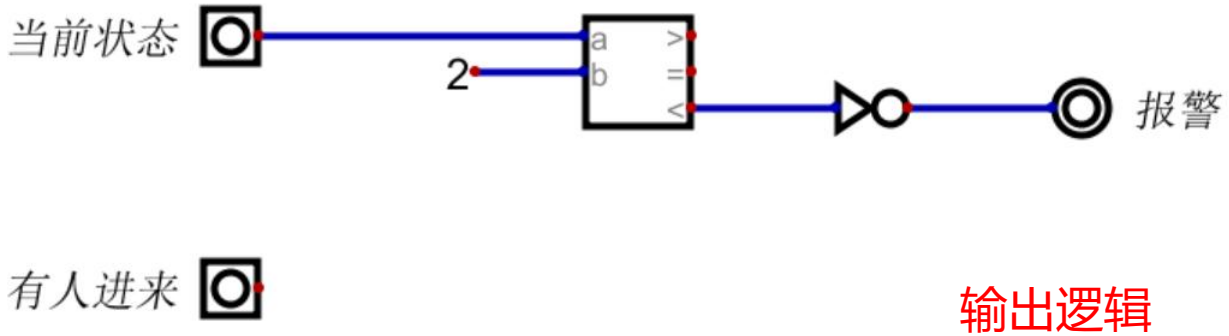


米利型FSM

时序逻辑示例1



下一状态逻辑



输出逻辑

时序逻辑示例2

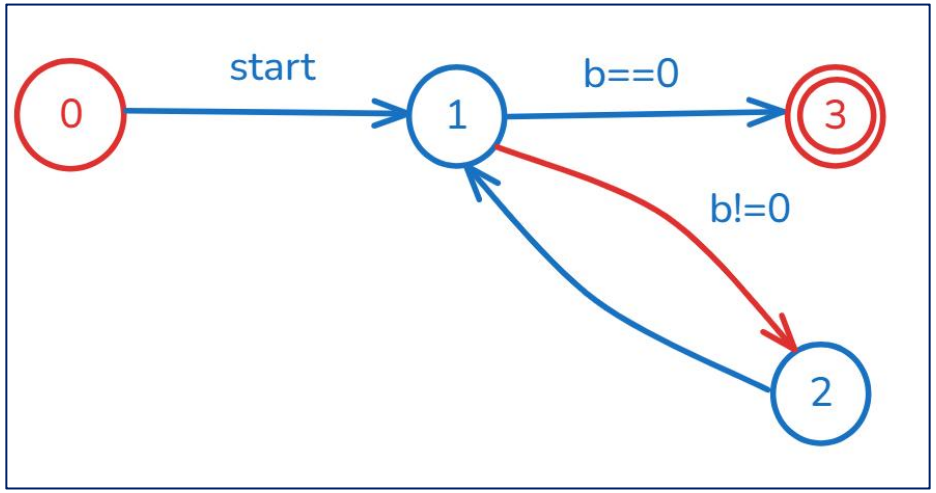
最大公因子

（欧几里得算法）

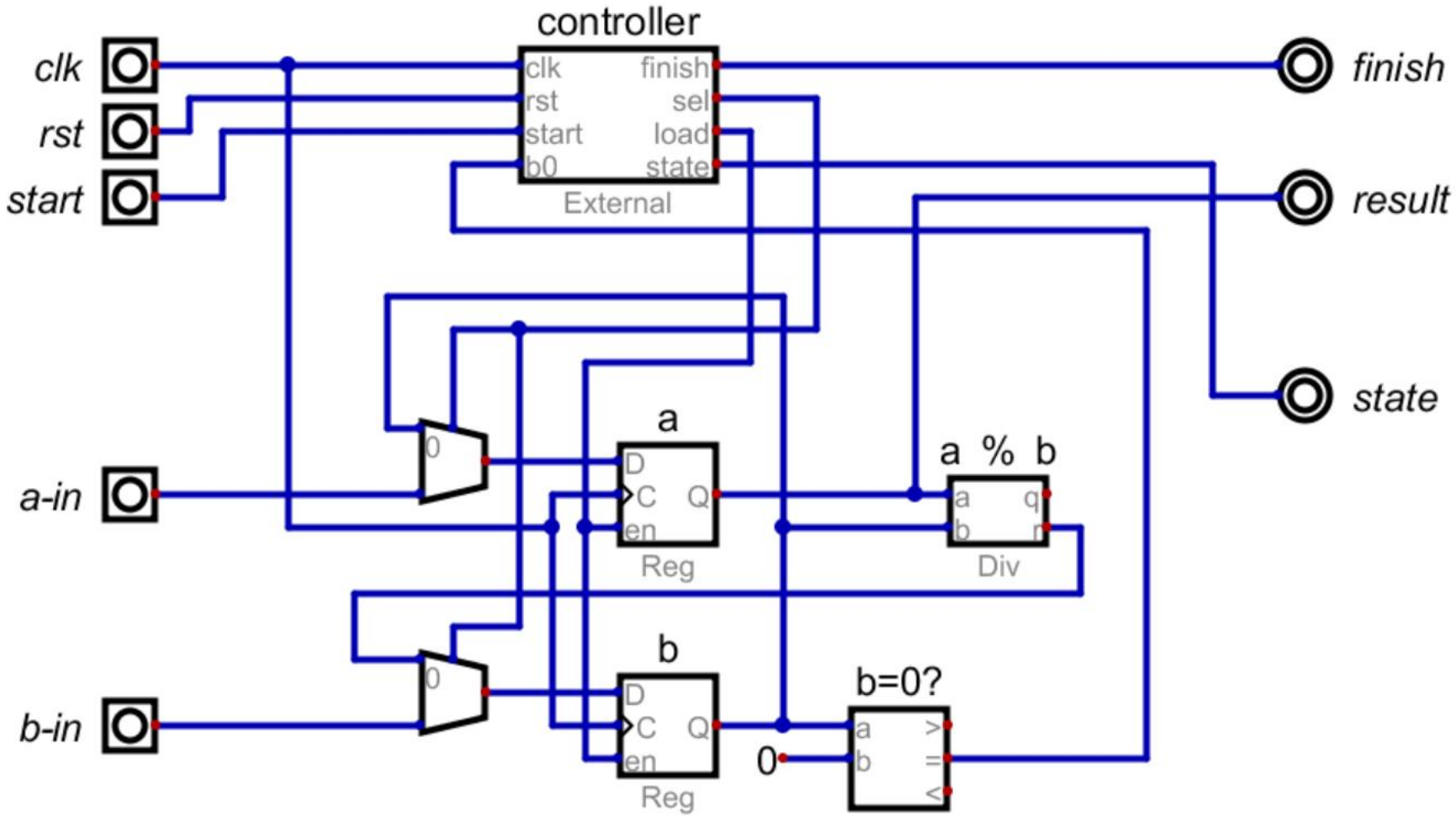
```
gcd(a, b):
    if b == 0:
        return a
    else:
        return gcd(b, a % b)
```

gcd	a	b	b = 0?	a % b
gcd(12,8)	12	8	假	4
gcd(8, 4)	8	4	假	0
gcd(4, 0)	4	0	真	

gcd(12, 8) = 4



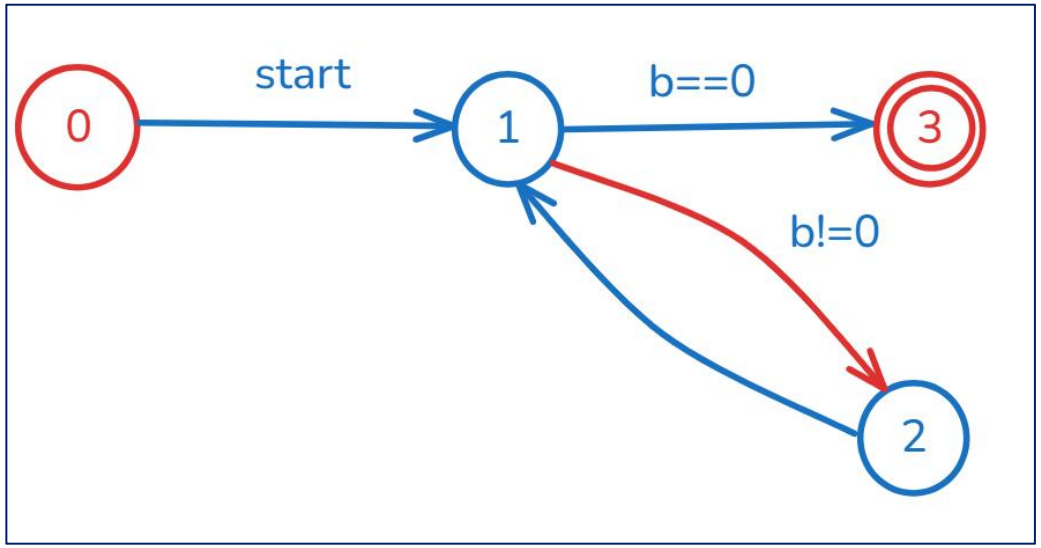
时序逻辑示例2


controller
clk
rst
start
b0
finish
sel
load
state
External
clk
rst
start
finish
result
state
a-in
b-in
0
a
D
C
Q
en
Reg
a % b
a
b
q
r
Div
b
b=0?
0
>
=
<

■ 时序逻辑示例2

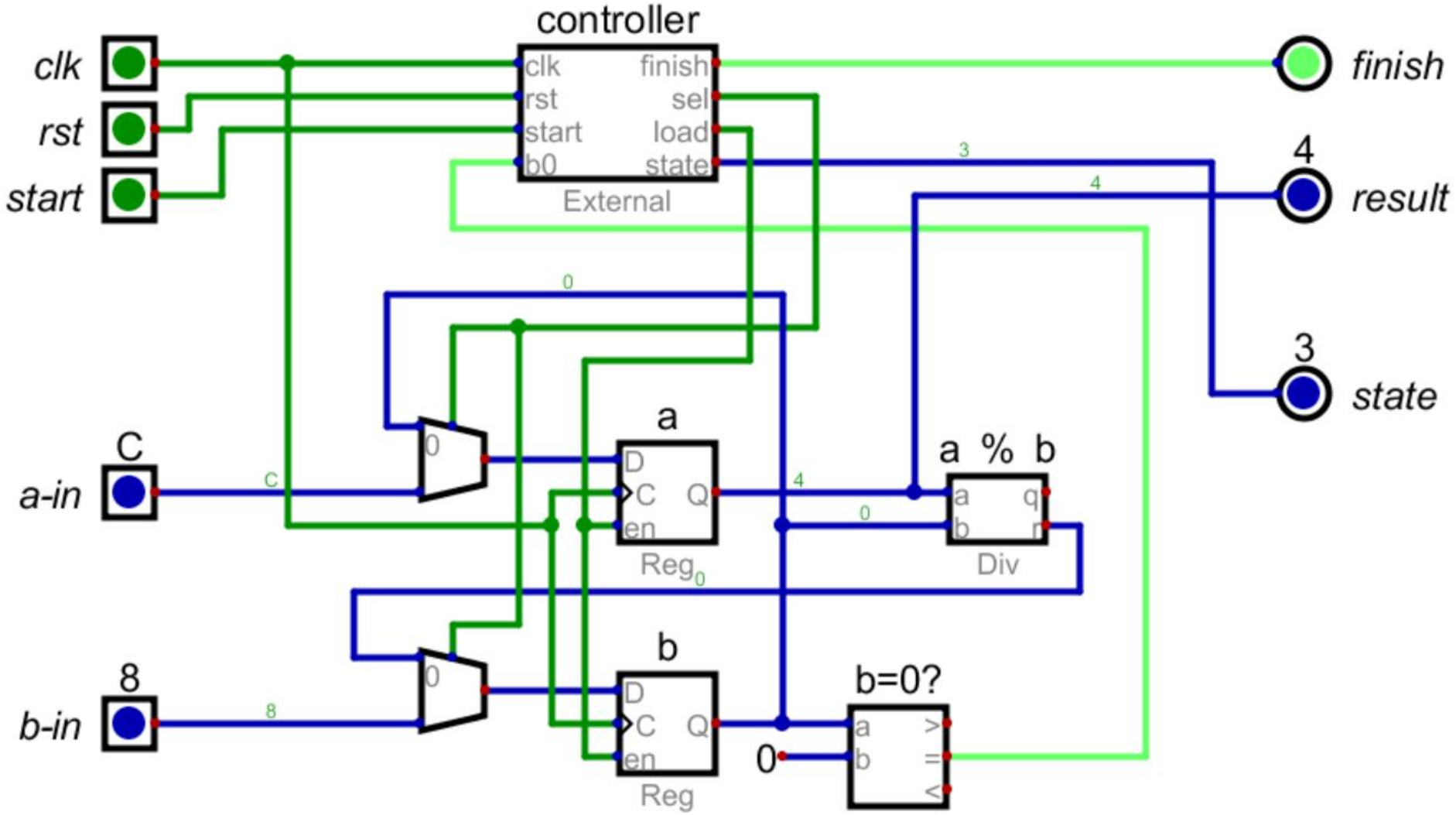
```
// 1. 当前状态
always @(posedge clk)
begin
        if(rst)
                state = 0;
        else
                state = next;
end
```

```
//3. 输出逻辑
assign finish = state == 3;
assign sel = state == 0;
assign load = state == 0 || state == 2;
```




```
// 2. 下一状态逻辑
always @(*)
begin
        case(state)
                0: begin
                        if(start)
                                next = 1;
                        else
                                next = 0;
                   end
                1: begin
                        if(b0)
                                next = 3;
                        else
                                next = 2;
                   end
                2: next = 1;
                3: next = 3;
                default: next = 0;
        endcase
end
```

时序逻辑示例2


controller
clk
rst
start
b0
finish
sel
load
state
External
finish
4
result
3
state
clk
rst
start
C
a-in
8
b-in
a
b
Reg0
Reg
a % b
Div
b=0?
0
3
4
0
c
8

RISC-V®

数字逻辑基础

- 布尔代数
- 数字逻辑
- 组合电路设计
- 时序电路设计